



**COMSATS University Islamabad,
Sahiwal Campus, Pakistan**

PlanLock – Personal Organizer Application

**A project presented to
COMSATS Institute of Information Technology, Sahiwal
Campus**

By

Urba Saleem	CIIT/FA22-BSE-066/SWL
Rabia Zafar	CIIT/FA22-BSE-079/SWL
Marrium Jamil	CIIT/FA22-BSE-096/SWL

Supervisor

Dr. Javed Ferzund

**In partial fulfillment
of the requirement for the degree of
Bachelor of Science in Software Engineering (2022-2026)**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Urba Saleem

Rabia Zafar

Marrium Jamil

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “**PlanLock - Personal Organizer Application**” was developed by **Urba Saleem (CII/FA22-BSE-066)**, **Rabia Zafar (CIIT/FA22-BSE-079)**, and **Marrium Jamil (CIIT/FA22-BSE-096)** under the supervision of **Dr. Javed Ferzund** and co supervisor **Mr. Arslan Sarwar** and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Dr. Javed Ferzund

Professor

External Examiner

Dr Riaz UL Amin

Associate Professor

Department of Computer Science

University of Okara

Head of Department

Department of Computer Science

Dr. Zafar Iqbal Roy

Assistant Professor

Executive Summary

In today's fast-paced digital environment, students and professionals often struggle to manage their daily tasks, habits, schedules, and personal goals efficiently. Most existing productivity tools focus on only one or two areas such as task management, note-taking, or habit tracking, forcing users to switch between multiple applications. This fragmented experience increases stress, reduces focus, and negatively affects consistency and productivity.

To address this problem, PlanLock has been proposed as an all-in-one intelligent personal organizer mobile application. Purpose is to combine multiple productivity features into a single, user-friendly platform that helps users plan, organize, and improve their daily routines. The system integrates task management, habit tracking, focus timer, note-taking, AI-powered recommendations, and community collaboration within one application.

PlanLock allows users to create and manage tasks with priorities, deadlines, and reminders. It also includes a habit tracker supported by streaks to motivate long-term consistency. A built-in focus timer helps users improve concentration and time management during study or work sessions. The note-taking module enables users to record important ideas. An intelligent recommendation system, which analyzes user activities and suggests relevant resources. In addition, community and collaboration features allow users to connect with others through shared goals, messages, and group activities.

The system is developed using Flutter for cross-platform mobile application development, Firebase for authentication and database services. These technologies ensure a responsive, secure, and scalable application with real-time synchronization.

In conclusion, PlanLock is designed to simplify digital organization by reducing the need for multiple applications and providing users with a centralized productivity solution. It promotes discipline, focus, motivation, and balanced personal and professional growth. This project also demonstrates practical implementation of software engineering concepts, mobile development, cloud computing, and user-centered design principles.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **“Dr. Javed Ferzund”** and our Co-Supervisor **“Mr. Arslan Sarwar”**. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Urba Saleem

Rabia Zafar

Marrium Jamil

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
CI/CD	Continuous Integration / Continuous Deployment
CURD	Create, Read, Update, Delete
DB	Database
DM	Direct Message
ERD	Entity Relationship Diagram
FAQ	Frequently Asked Questions
FCM	Firebase Cloud Messaging
FR	Functional Requirement
GB	Gigabyte
GUI	Graphical User Interface
IDE	Integrated Development Environment
MB	Megabyte
NFR	Non-Functional Requirement
OTP	One-Time Password
OS	Operating System

PC	Personal Computer
PDF	Portable Document Format
RAM	Random Access Memory
REST	Representational State Transfer
SDK	Software Development Kit
SDD	Software Design Description
SDLC	Software Development Life Cycle
SQL	Structured Query Language
SRS	Software Requirement Specification
UI	User Interface
UX	User Experience
UML	Unified Modeling Language
XP	Experience Points

Table of Contents

1. Introduction	2
1.1 Brief	2
1.2 Relevance of Course Modules.....	3
1.3 Project Background.....	3
1.4 Literature Review	4
1.5 Analysis from Literature Review	5
1.6 Methodology and Software Lifecycle for This Project	6
1.6.1 Rationale behind Selected Methodology	6
2. Problem Definition	8
2.1 Problem Statement	8
2.2 Deliverables and Development Requirements	9
2.2.1 Deliverables	9
2.2.2 Development Requirements	10
2.3 Current System.....	11
3. Requirement Analysis	13
3.1 Use Cases Diagram(s)	13
3.2 Detailed Use Cases.....	15
3.2.1 User Authentication	15
3.2.2 Manage Tasks	16
3.2.3 Track Habits.....	17
3.2.4 Earn Streak.....	18
3.2.5 AI Recommendations.....	20
3.2.6 Focus Tracker with Timer	21
3.2.7 Note-Taking	22
3.2.8 Community & Collaboration	23
3.3 Functional Requirements.....	24
3.3.1 Authenticate Users	24
3.3.2 Manage Tasks	25
3.3.3 Track Habits.....	25
3.3.4 Calculate Streaks and Rewards	26
3.3.5 Give AI Recommendations	27
3.3.6 Allow Focus Tracking with Timer	28
3.3.7 Allow Note-Taking	29
3.3.8 Allow Collaboration with Other Users	30
3.3.9 Allow Theme Controls.....	31
3.4 Non-Functional Requirements	31

3.4.1 Usability	32
3.4.2 Performance	32
3.4.3 Reliability & Availability	33
3.4.4 Security	33
3.4.5 Scalability	33
4. Design and Architecture.....	36
4.1 System Architecture	36
4.1.1 Architectural Design	36
4.2 Data Representation	38
4.2.1 Data Dictionary	38
4.3 Process Flow Representation.....	46
4.4 Design Model	47
4.4.1 Class Diagram.....	47
4.4.2 Sequence Diagram	49
4.4.3 State Transition Diagram	50
4.4.4 Data Flow Diagram	51
5. Implementation.....	56
5.1 Algorithm	56
5.2 External APIs	61
5.3 User Interface	62
6. Testing and Evaluation.....	86
6.1 Manual Testing.....	86
6.1.1. System Testing.....	86
6.1.2 Unit Testing	87
6.1.3 Functional Testing	89
6.1.4 Integration Testing	90
6.2 Automated Testing	92
6.3 Tools Used	94
7. Conclusion and Future Work	97
7.1 Conclusion.....	97
7.2 Future Work	97
8. References.....	99

List of Figures

Figure 3.1: PlanLock Use Case Diagram.....	14
Figure 4.1.1: PlanLock Architecture Diagram	37
Figure 4.2.1: PlanLock Entity Relationship Diagram.....	45
Figure 4.4.1: PlanLock Class Diagram	48
Figure 4.4.2: PlanLock Sequence Diagram	49
Figure 4.4.3: PlanLock State Transition Diagram	50
Figure 4.4.4-1: PlanLock Data Flow Diagram (Level 0)	51
Figure 4.4.4-2: PlanLock Data Flow Diagram (Level 1)	52
Figure 4.4.4-3: PlanLock Data Flow Diagram (Level 2)	53
Figure 4.4.4-4: PlanLock Data Flow Diagram (Level 3)	54
Figure 5.3.1: Login Screen.....	63
Figure 5.3.2: Signup Screen.....	64
Figure 5.3.3: Reset Password Screen.....	65
Figure 5.3.4: Reset Password Link Email Screen.....	66
Figure 5.3.5: Profile Screen	67
Figure 5.3.6: Edit Profile Screen.....	68
Figure 5.3.7: Home Screen	69
Figure 5.3.8: Task Screen	70
Figure 5.3.9: Add Task Dialog.....	71
Figure 5.3.10: Habit Tracker Page.....	72
Figure 5.3.11: Add Habit Dialog	73
Figure 5.3.12: Habit Tracker Calendar View Page.....	74
Figure 5.3.13: Habit Tracker Custom Habits Page	75
Figure 5.3.14: Notes Page.....	76
Figure 5.3.15: Notes Editing View	77
Figure 5.3.16: AI recommendation Page	78
Figure 5.3.17: Focus Tracker Page	79
Figure 5.3.18: Focus Tracker Calendar View	80
Figure 5.3.19: Community Page	81
Figure 5.3.20: Community Project Tasks	82
Figure 5.3.22: Setting Page.....	83
Figure 5.3.23: Dark Theme Page	84

List of Tables

Table 1.5: Related System Analysis with proposed project solution.....	5
Table 3.2.1: User Authentication	15
Table 3.2.2: Create and Manage Tasks	16
Table 3.2.3: Create and Track Habits	17
Table 3.2.4: Award Streaks.....	19
Table 3.2.5: Generate Personalized Recommendations	20
Table 3.2.6: Start and Monitor Focus Sessions.....	21
Table 3.2.7: Create and Manage Notes	22
Table 3.2.8: Participate in Community Activities.....	23
Table 3.3.1: User Authentication	24
Table 3.3.2: Task Management.....	25
Table 3.3.3: Habit Tracker	26
Table 3.3.4: Calculate Streaks and Rewards.....	27
Table 3.3.5: AI recommendation	28
Table 3.3.6: Focus Tracking with Timer.....	29
Table 3.3.7: Note-Taking	30
Table 3.3.8: Community and Collaboration.....	31
Table 4.2.1: PlanLock Data Dictionary	39
Table 6.1.1: System Testing.....	86
Table 6.1.2: Unit Testing	87
Table 6.1.3: Functional Testing	89
Table 6.1.4: Integegration Testing	91
Table 6.1.5: Automated Testing.....	93
Table 6.3: Testing Tools and their Usage	94

Chapter 1

INTRODUCTION

1. Introduction

PlanLock is a modern personal organizer mobile application designed to simplify planning, organizing, and managing daily tasks and habits. In today's fast-paced world, individuals often face stress, missed deadlines, and the frustration of using multiple fragmented digital tools. Constantly switching between apps for task management, habit tracking, note-taking, and collaboration adds unnecessary complexity. PlanLock addresses this problem by combining essential features into a single, cohesive platform. The system offering personalization for managing personal and professional routines within a single profile. Core modules include task management, habit tracking with streaks, AI-powered recommendations, and community collaboration. Developed using Flutter for Android, with Firebase. The system leverages psychological motivators such as streaks, rewards, and progress visualization to encourage long-term consistency and engagement. By minimizing digital overload and merging planning, habit tracking, and collaboration into a single intuitive platform, PlanLock enhances users' quality of life, reduces stress and fosters long-term growth.

1.1 Brief

PlanLock is an intelligent personal organizer mobile application designed to help users manage their daily tasks, track habits, take notes, and collaborate within a single platform. It allows users to organize and prioritize tasks, set reminders, and monitor progress, while habit tracking features such as streaks, and achievements encourage consistency. PlanLock solves this problem by integrating all these essential features into a single user-friendly application.

The system also includes a focus timer, calendar view, progress graphs, and AI-based recommendations, providing a comprehensive solution for improving productivity and time management.

The application is developed for Android using Flutter for the front end and Firebase for backend and cloud services, following Object-Oriented Design (OOD) and the Agile development lifecycle. The initial version focuses on core features like task management, habit tracking, and AI recommendations, while future enhancements include real-time collaboration, advanced AI capabilities, and third-party integrations. [1]

1.2 Relevance of Course Modules

The development of PlanLock is closely related to the core courses studied in the Software Engineering (SE) program, as it applies both theoretical knowledge and practical development skills. The project strongly relates to Software Engineering, where concepts such as requirement gathering, Software Requirement Specification (SRS), Software Design Description (SDD), software development life cycle models, testing, and project management are applied throughout the development process. It also involves Object-Oriented Programming, where classes, objects, inheritance, encapsulation, and modular programming techniques are used to build a structured and maintainable system.

The project is connected to Database Systems through the design and management of databases using Firebase for secure storage of user accounts, tasks, habits, and progress records. Knowledge of Mobile Application Development is utilized in building the Android application using the Flutter framework, ensuring a responsive and cross-platform user experience. Concepts of Human Computer Interaction (HCI) are applied to design a clean, intuitive, and user-friendly interface that improves usability and accessibility.

PlanLock also relates to Artificial Intelligence, as AI-based recommendation features are integrated to enhance productivity and user engagement. In addition, Computer Networks concepts are used for cloud synchronization, API communication, and secure internet-based data transfer. Finally, Data Structures and Algorithms play an important role in efficient data handling, task organization, search operations, and overall system performance. Therefore, PlanLock combines knowledge from multiple course modules into one complete and meaningful final year project.

1.3 Project Background

With the increasing demands of academic, professional, and personal life, individuals often struggle to manage their tasks, maintain productive habits, and stay organized. Most existing solutions focus on a single aspect such as task management, note-taking, or habit tracking, forcing users to rely on multiple applications. This not only creates fragmentation but also reduces efficiency and increases cognitive load. The idea behind PlanLock is to provide a

unified productivity platform that combines all these essential features into one system, allowing users to manage their daily activities more effectively and in a structured manner.

PlanLock works by integrating core productivity components, such as task management, habit tracking, note-taking, and focus tools into a single application supported by AI-driven assistance. Users can create and track tasks, build consistent habits, store notes, and receive personalized recommendations based on their activities. The system leverages modern mobile development technologies and cloud services to ensure real-time data synchronization, secure access, and scalability. By combining these features, PlanLock aims to simplify planning, improve time management, and enhance overall productivity.

1.4 Literature Review

Recent trends in productivity applications focus on AI-based assistance, gamification, and all-in-one digital workspace solutions that combine multiple productivity features into a single platform. However, most existing systems still lack a balanced integration of simplicity, intelligence, and comprehensive functionality.

Todoist: Todoist is one of the leading task management platforms, offering robust task creation, categorization and project management. However, it lacks built-in habit tracking and AI recommendations, limiting its scope for holistic productivity management. [2]

Habitica: Habitica turns habit tracking into a role-playing game where users earn rewards for completing tasks and building habits. While it excels in gamification, it lacks a strong task management system and professional productivity features needed for users with complex workflows.

Notion: Notion is a flexible workspace that combines notes, databases, and project management. Although highly customizable, Notion can be overwhelming to new users due to its steep learning curve and lack of built-in automation for habit tracking or AI recommendations. [3]

Google Keep: These note-taking tools help users save ideas and organize notes. However, they focus mainly on notes and do not integrate tasks, habits, or productivity tracking in a complete way.

The related systems provide a foundation in task management and habit tracking, but there remains a gap in offering an integrated, intelligent, and easy-to-use solution for users who want to manage their personal development and daily productivity in one place.

1.5 Analysis from Literature Review

The analysis of existing systems highlights that no single application fully satisfies all productivity needs. Todoist excels in task management but lacks habit tracking and intelligent assistance. Habitica improves user engagement through gamification but is not suitable for structured professional task handling. Notion provides flexibility but becomes complex and lacks automation features for productivity enhancement.

From the analysis given in Table 1.5, it is clear that users require a system that combines simplicity, intelligence, and integration. PlanLock addresses these limitations by offering a unified platform that integrates task management, habit tracking, note-taking, and AI-based recommendations in a single application. Unlike existing systems, it focuses on ease of use, automation, and productivity enhancement, making it more suitable for students and professionals who want an efficient daily management solution.

Table 1.5: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
Todoist	Lacks habit tracking and AI recommendations	PlanLock will integrate both task management and habit tracking with AI-based content suggestions
Habitica	No task management for professional tasks	PlanLock will offer professional-grade task management alongside habit tracking
Notion	Overwhelming interface, lack of built-in productivity features	PlanLock will provide a simple, minimalist interface with built-in habit management and smart task recommendation.

1.6 Methodology and Software Lifecycle for This Project

The development of PlanLock follows the Object-Oriented Design (OOD) methodology, which emphasizes modularity, reusability, and scalability. By modeling the system as a collection of interacting objects such as Users, Tasks, Habits, Notes, and AI modules—OOD allows for a clear representation of real-world entities and their behaviors. This approach simplifies maintenance, enables code reuse across different modules, and supports future expansion, such as integrating new features like advanced AI or real-time collaboration. Each module in PlanLock, including task management, habit tracking, note-taking, and community collaboration, is designed as an independent object with defined attributes and methods, promoting separation of concerns and reducing system complexity.

For the software development process, PlanLock follows the Agile methodology, which supports iterative and incremental development. Agile allows the team to deliver functional components in small cycles, gather continuous user feedback, and adapt to evolving requirements. Combining OOD with Agile ensures a flexible, maintainable, and quality-focused development process, allowing PlanLock to evolve efficiently while meeting user expectations.

1.6.1 Rationale behind Selected Methodology

The selection of Object-Oriented Design (OOD) for PlanLock is based on the need to efficiently manage a system composed of multiple modules such as Users, Tasks, Habits, and AI components. OOD enables modularity, reusability, and clear structure by representing real-world entities as objects, making the system easier to develop, maintain, and expand. It also supports encapsulation and abstraction, which reduce complexity and allow new features to be added with minimal impact on existing components, making it more suitable than traditional structural approaches for a scalable application.

The adoption of the Agile software development lifecycle is justified by the project's need for flexibility and continuous improvement. Agile supports iterative development, allowing features like habit tracking and AI recommendations to be built, tested, and refined in cycles with regular user feedback. This ensures better quality, faster adaptation to changing requirements, and a final product that closely aligns with user needs, making Agile an ideal choice for PlanLock.

Chapter 2

PROBLEM DEFINITION

2. Problem Definition

The PlanLock system provides a unified platform for planning and organization, integrating task management, habit tracking, note-taking, and collaboration. Users can create, edit, categorize, and prioritize tasks, set due dates, reminders, and difficulty levels, and track their progress with search functionality. Habit tracking helps users monitor daily, weekly, or monthly routines, offering visual streaks, XP points, achievement badges, and notifications to encourage consistency. A built-in calendar and progress graphs provide a clear overview of schedules and achievements, reducing stress from managing multiple apps. Streaks, and achievements further motivate users to stay engaged, while an AI-based recommendation system offers personalized resources tailored to individual habits. The platform ensures secure user authentication, cloud synchronization, notifications, and privacy controls for achievements and streaks.

Additional features include a focus timer that allows timed work or study sessions, logging focus levels and generating progress charts to improve time management. The note-taking module provides a space for creating and organizing notes through typing, and optional community sharing. Community collaboration enables messaging, project creation, shared task views, and profile sharing while respecting privacy settings. PlanLock is implemented for Android mobile devices using Flutter for the front end, Firebase for cloud services. The initial implementation focuses on single-user task management, habit tracking, and AI recommendations, with future scalability planned for real-time collaboration, deep third-party integrations, and advanced AI features.

2.1 Problem Statement

Managing work, studies, and personal routines has become overwhelming, with planning tools scattered across multiple apps. Existing apps provide partial solutions but often lack intelligent features that adapt to user needs. Switching between different apps cause distraction, stress and frustrated when deadlines are missed due to poor organization. This fragmented experience wastes time, affects consistency, leaves habits untracked. Many existing apps provide partial solutions, but they rarely combine task planning, habit tracking, AI recommendation, and collaboration within one cohesive system.

PlanLock addresses this problem by offering an all-in-one intelligent productivity platform that integrates task management, habit tracker, focus tracker, note-taking, AI recommendations, and community collaboration in a unified space. It enhances focus, and helps users maintain balance in their daily routines. While apps like Habitica, Todoist, and Notion, lack integration of AI-driven suggestions, deep habit tracking in a minimalist yet effective design.

Developing this system enhances understanding of mobile application development, cloud synchronization, AI integration, and user-centered design principles. It also strengthens technical expertise in Flutter, Firebase, database management, responsive UI/UX development and data analytics. This project provides an opportunity to apply theoretical knowledge in a practical environment, promoting a deeper understanding of the software development lifecycle (SDLC) and real-world problem solving while contributing an innovative, meaningful solution to modern digital organization challenges. [4]

2.2 Deliverables and Development Requirements

2.2.1 Deliverables

The main deliverable of the PlanLock project will be a fully functional Android mobile application developed using Flutter. The application will provide an all-in-one platform for task management, habit tracking, note-taking, focus timing, AI recommendations, and community collaboration. Users will be able to manage both personal and professional activities through a single account with personalized dashboards.

Another important deliverable, the system will include a secure authentication module that allows users to register, log in, reset passwords, and manage profiles. Firebase Authentication will be used to ensure secure access and data privacy. This module will also support user session management for seamless login experiences.

PlanLock will deliver an integrated module for task management and habit tracking. Users will be able to create tasks, assign deadlines, set reminders, organize priorities, and track recurring habits. The module will also include streak tracking. Project will provide AI-powered recommendations based on user activities, tasks, and habits. It will suggest useful resources such as articles, videos, and productivity tips

PlanLock will include community features that allow users to communicate, share progress, and collaborate on group projects. Users can interact through direct messages, community groups, and shared productivity challenges, increasing motivation and teamwork. A focus timer will allow users to start focus sessions, record study or work hours.

The project will also deliver complete academic and technical documentation including the Software Requirement Specification (SRS), Software Design Description (SDD), testing reports, user manual, presentation slides, and final report. These documents will support implementation, maintenance, and evaluation of the system.

2.2.2 Development Requirements

The development of PlanLock requires Android Studio as the main IDE, Flutter SDK for mobile application development, and Firebase services for backend operations. UI/UX design tools such as Figma or Canva will be used for interface prototyping. [5]

The project also requires computers with sufficient processing power, Android mobile devices for testing, a stable internet connection will be required for cloud synchronization, authentication, and AI recommendations. In addition, successful development depends on effective teamwork, task division, and supervision throughout the project lifecycle.

2.3 Current System

At present, users depend on multiple separate applications to manage productivity-related activities. Task management is often handled through apps like Todoist, habit tracking through Habitica or Loop Habit Tracker, and note-taking through platforms such as Notion, Google Keep, or Evernote. Since these systems operate independently, users must frequently switch between apps to manage their daily routines, which increases complexity and reduces convenience.

Most current applications focus only on one specific feature and do not provide a complete productivity solution. Many systems lack integrated AI-based recommendations, productivity coaching, forcing users to rely on external tools for guidance and support. In addition, several personal productivity applications provide limited collaboration features, making it difficult for users to share goals, work in teams, or monitor group progress effectively.

The major weakness of the current system is fragmentation. Tasks, habits, notes, reminders, timers, and collaboration tools are distributed across multiple platforms, resulting in wasted time, distractions, and poor workflow management. Furthermore, storing personal data across different applications with separate privacy policies creates security concerns and reduces user control over information. Therefore, the current system does not fully meet the needs of modern students and professionals. PlanLock is proposed to overcome these limitations by offering one integrated, intelligent, and user-friendly productivity platform.

Chapter 3

REQUIREMENT ANALYSIS

3. Requirement Analysis

The development of PlanLock follows the Object-Oriented Design (OOD) methodology, which emphasizes modularity, reusability, and scalability. By modeling the system as a collection of interacting objects—such as Users, Tasks, Habits, Notes, and AI modules—OOD allows for a clear representation of real-world entities and their behaviors. This approach simplifies maintenance, enables code reuse across different modules, and supports future expansion, such as integrating new features like advanced AI or real-time collaboration. Each module in PlanLock, including task management, habit tracking, note-taking, and community collaboration, is designed as an independent object with defined attributes and methods, promoting separation of concerns and reducing system complexity.

For the software development process, PlanLock follows the Agile methodology, which supports iterative and incremental development. Agile allows the team to deliver functional components in small cycles, gather continuous user feedback, and adapt to evolving requirements. This approach is particularly suitable for PlanLock, as it ensures that core modules—like habit tracking, AI recommendations, and focus timers—can be tested, refined, and improved before full-scale deployment. Combining OOD with Agile ensures a flexible, maintainable, and quality-focused development process, allowing PlanLock to evolve efficiently while meeting user expectations.

3.1 Use Cases Diagram(s)

Figure 3.1 is the use case diagram for the PlanLock system. It shows all the basic tasks that the user will perform in the app (adding to-do lists, adding hobbies to track, interacting with other users in community). It shows key functionalities such as user authentication, task and habit management, AI recommendations, focus tracking, note-taking, and community interaction. The diagram also includes extended and included use cases like password recovery, reminders, streaks & XP tracking, and calendar view, which enhance the overall user experience.

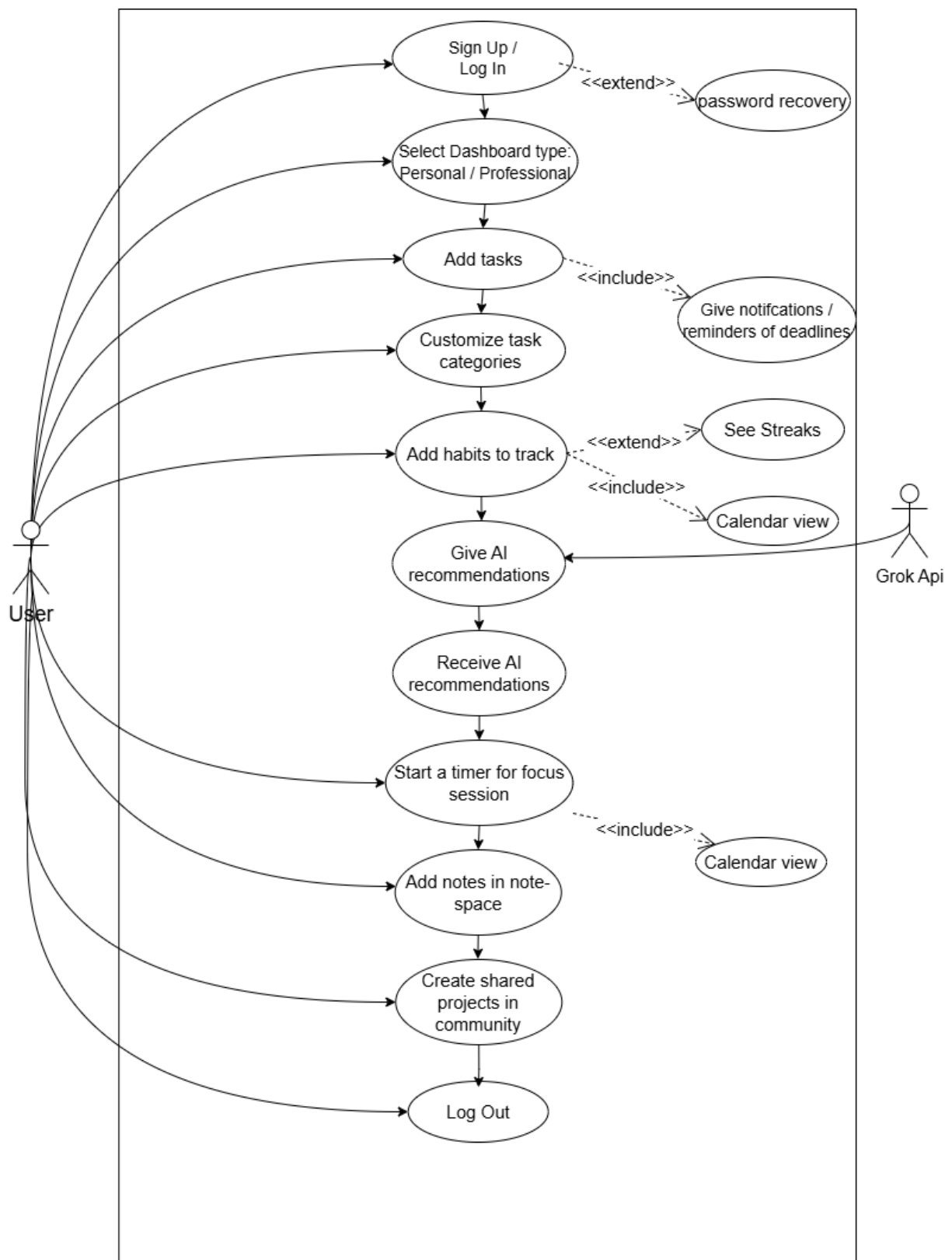


Figure 3.1: PlanLock Use Case Diagram

3.2 Detailed Use Cases

This section presents the detailed use cases of the PlanLock system. Each use case describes a specific interaction between the user and the system, including the actors involved, preconditions, normal flow, alternative flow, postconditions, and business rules. These use cases help in understanding how different modules of the application will function and how users will interact with the system.

3.2.1 User Authentication

In Table 3.2.1, User Authentication allows users to securely register, log in, and manage their profiles. It protects personal data through encrypted credentials and secure verification methods. The module also maintains user sessions for smooth access across multiple devices. It ensures that only authorized users can access private productivity data and personalized features. This process builds trust, security, and convenience for users.

Table 3.2.1: User Authentication

Use Case ID	UC1
Use Case Name	User Authentication
Actors	Primary: Users Secondary: Authentication Service, Database
Description	A user registers or logs in to the PlanLock app to access personalized features such as tasks, habits, and streak tracking.
Trigger	User opens the app and selects “Login” or “Sign Up.”
Preconditions	PRE-1. The app must be installed and running. PRE-2. User has a stable internet connection.
Postconditions	POST-1. User account is verified and stored in the database. POST-2. User is redirected to the home dashboard.

Normal Flow	1. User opens the app and selects “Sign Up” or “Login.” 2. System displays authentication options (email/password or social login). 3. User enters credentials and submits. 4. System validates credentials with the authentication service. 5. On success, system fetches user data from the database. 6. System displays personalized home dashboard.
Alternative Flows	1.1. Invalid credentials entered → System displays “Invalid credentials” message and allows retry. 1.2. Forgot password → User selects “Forgot Password,” enters email, and resets via OTP/email link.
Exceptions	Authentication service unavailable → System notifies user and retries after delay.
Business Rules	BR-1: Passwords must be at least 8 characters long. BR-2: Email must be unique.
Assumptions	Assume the majority of users register via email/password.

3.2.2 Manage Tasks

In Table 3.2.2, this use case enables users to create, edit, organize, and delete daily or long-term tasks. It supports priority levels, deadlines, reminders, and completion tracking for better productivity management. Users can sort tasks based on urgency or category for easier planning. The task system helps users remain organized and focused on personal or professional goals. It acts as one of the core productivity modules of PlanLock.

Table 3.2.2: Create and Manage Tasks

Use Case ID	UC2
Use Case Name	Create and Manage Tasks
Actors	Primary: User Secondary: Task Management System, Notification Service

Description	A user creates, edits, and manages personal or professional tasks to stay organized.
Trigger	User selects the “Add Task” or “Edit Task” button.
Preconditions	PRE-1. User must be logged in.
Postconditions	POST-1. New or updated task is saved in the database. POST-2. Notifications are scheduled if applicable.
Normal Flow	1. User selects “Add Task.” 2. System prompts for task title, description, deadline, and category. 3. User fills details and clicks “Save.” 4. System stores the task and displays it in the task list. 5. User can later mark the task as completed or delete it.
Alternative Flows	2.1. User edits an existing task → Updates fields and saves. 2.2. User deletes a task → System confirms deletion and removes it.
Exceptions	Task save fails due to network error → System shows retry option.
Business Rules	BR-1 Task title is mandatory. BR-2 Each task must belong to a category (Daily, Weekly, Monthly).
Assumptions	Assume each user manages up to 100 active tasks at once.

3.2.3 Track Habits

In Table 3.2.3, the system allows users to create and monitor recurring routines such as exercise, reading, or meditation. The system records progress regularly and displays consistency using charts and statistics. It motivates users to build positive routines through reminders and streak counts. Habit history helps users analyze improvement over time.

Table 3.2.3: Create and Track Habits

Use Case ID	UC3
Use Case Name	Create and Track Habits

Actors	Primary: User Secondary: Habit Tracker Module
Description	User adds habits to track daily progress such as exercise, reading, or meditation.
Trigger	User selects “Add Habit.”
Preconditions	PRE-1. User must be logged in.
Postconditions	POST-1. Habit is added and visible in daily tracker view.
Normal Flow	1. User clicks “Add Habit.” 2. System prompts for habit name and frequency. 3. User enters details and confirms. 4. System saves habit and displays it in habit list. 5. User marks habits as complete each day.
Alternative Flows	3.1 User edits habit details (name/frequency). 3.2 User deletes a habit.
Exceptions	Habit save fails due to sync issue → System retries.
Business Rules	BR-1 User can track a maximum of 10 habits simultaneously.
Assumptions	Assume user data syncs daily.

3.2.4 Earn Streak

In Table 3.2.4, this use case rewards users with streak counts, badges, and achievement levels for completing habits consistently. It introduces gamification elements that make productivity more engaging and enjoyable. Users are encouraged to maintain routines and avoid missing progress targets. On achieving certain levels, the user receives badges, and these levels are based on the number of streaks the user has maintained. Milestones and rewards increase motivation, commitment, and long-term engagement with the system.

Table 3.2.4: Award Streaks

Use Case ID	UC4
Use Case Name	Award Streaks and rewards for Progress
Actors	Primary: User Secondary: Streaks / rewards Engine
Description	User earns streak counts, achievement levels, and badges by maintaining habits regularly.
Trigger	User completes a habit.
Preconditions	PRE-1. At least one habit is active.
Postconditions	POST-1. Streak count is updated. POST-2. Achievement level and badges are awarded when milestone streak targets are reached.
Normal Flow	1. User completes a habit. 2. System records completion. 3. Streak count increases. 4. System checks if a new level is reached based on total streaks. 5. If eligible, system awards badge and updates achievement level. 6. User receives reward notification.
Alternative Flows	4.1 User misses required streak → System resets current streak count.
Exceptions	XP update fails due to offline mode → System queues update for next sync.
Business Rules	BR-1: Levels are determined by number of streaks achieved. BR-2: Badges are unlocked only when defined streak milestones are completed.

Assumptions	Assume streak resets if no required activity occurs within the scheduled time period.
-------------	---

3.2.5 AI Recommendations

In Table 3.2.5, the AI recommendation feature studies user activity, schedules, and productivity patterns to provide personalized suggestions. It recommends better habits, task priorities, useful resources, and motivational tips. Over time, the AI learns user preferences and becomes more accurate. This feature reduces decision-making effort and improves planning efficiency. It adds intelligence and personalization to the platform.

Table 3.2.5: Generate Personalized Recommendations

Use Case ID	UC5
Use Case Name	Generate Personalized Recommendations
Actors	Primary: User Secondary: AI Engine
Description	AI recommends productivity strategies, task priorities, and motivational prompts based on user behavior.
Trigger	User opens “AI Insights” tab or system triggers daily analysis.
Preconditions	PRE-1. User has at least one week of activity data.
Postconditions	POST-1. AI suggestions are generated and displayed.
Normal Flow	1. User navigates to “AI Insights.” 2. System analyzes usage and focus patterns. 3. AI generates personalized suggestions. 4. User views or applies recommendations.
Alternative Flows	5.1 User requests refresh → AI regenerates insights.
Exceptions	Insufficient data → System displays “Not enough data for insights.”

Business Rules	BR-1 AI requires minimum 7 days of user activity.
Assumptions	AI learns and improves over time.

3.2.6 Focus Tracker with Timer

In Table 3.2.6, the module provides a Pomodoro-style timer for focused work sessions and break management. Users can start, pause, or customize sessions according to their workflow. The system records focus time and displays reports for productivity analysis. It helps reduce distractions and improve concentration during study or work. Structured sessions promote better time management habits.

Table 3.2.6: Start and Monitor Focus Sessions

Use Case ID	UC6
Use Case Name	Start and Monitor Focus Sessions
Actors	Primary: User Secondary: Focus Timer Module
Description	User starts a focus timer for productivity (e.g., 25-minute sessions).
Trigger	User selects “Start Focus Session.”
Preconditions	PRE-1. User must be logged in.
Postconditions	POST-1. Focus session data is logged.
Normal Flow	1. User starts focus timer. 2. System starts countdown. 3. At completion, system notifies user to take a break. 4. System logs focus duration.
Alternative Flows	6.1 User pauses or ends session early → System saves partial data.
Exceptions	Timer interrupted by app crash → System restores previous state.

Business Rules	BR-1 Default focus duration is 25 minutes.
Assumptions	Assume user will complete at least one session per day.

3.2.7 Note-Taking

In Table 3.2.7, the Note Taking feature enables users to write, edit, and organize personal or work-related notes. Notes may be linked with tasks, habits, or projects for better context. Search and tagging options help users quickly retrieve important information. It serves as a digital workspace for capturing ideas and reminders. This feature increases convenience by keeping everything in one app.

Table 3.2.7: Create and Manage Notes

Use Case ID	UC7
Use Case Name	Create and Manage Notes
Actors	Primary: User Secondary: Notes Module
Description	User writes, edits, and organizes notes linked to tasks or general thoughts.
Trigger	User selects “New Note.”
Preconditions	PRE-1. User must be logged in.
Postconditions	POST-1. Note is saved and retrievable.
Normal Flow	1. User clicks “New Note.” 2. System opens editor. 3. User writes note and saves it. 4. Note appears in user’s note list.
Alternative Flows	7.1 User edits or deletes a note.
Exceptions	Note save fails → Retry option provided.

Business Rules	BR-1 Notes are private by default.
Assumptions	Assume users may attach notes to specific tasks.

3.2.8 Community & Collaboration

In Table 3.2.8, the system enables users to communicate, share progress, and collaborate on tasks. It supports group discussions, shared goals, and community engagement. This enhances teamwork and accountability.

Table 3.2.8: Participate in Community Activities

Use Case ID	UC8
Use Case Name	Participate in Community Activities
Actors	Primary: User Secondary: Community Server, Other Users
Description	Users interact through posts, comments, and collaborative challenges.
Trigger	User selects “Community” tab.
Preconditions	PRE-1. User must be logged in.
Postconditions	POST-1. Post or interaction is stored and displayed to others.
Normal Flow	1. User navigates to community. 2. System displays posts and activities. 3. User creates a post or comment. 4. System updates feed and notifies other users.
Alternative Flows	8.1 User joins a group challenge. 8.2 User reports inappropriate content.
Exceptions	Server timeout → User notified and retry suggested.
Business Rules	BR-1 Posts must comply with community guidelines.
Assumptions	Assume content is moderated automatically.

3.3 Functional Requirements

Functional requirements define the core operations and services that PlanLock must perform. They describe what the system should do in response to user actions. These requirements ensure that all important features are implemented correctly. They also serve as a foundation for development, testing, and system validation.

3.3.1 Authenticate Users

In Table 3.3.1, the system shall allow users to register, log in, and reset their passwords securely. Users will also choose between personal and professional profiles during registration. Incorrect credentials will trigger an error message, prompting the user to try again.

Table 3.3.1: User Authentication

Field	FR1
Title	User Authentication
Requirement	• FR1.1: Users must be able to sign up using email/password or third-party logins (e.g., Google). • FR1.2: System must validate credentials before granting access. • FR1.3: Passwords must be stored securely (hashed and salted). • FR1.4: Users should be able to reset forgotten passwords via email verification. • FR1.5: Only authenticated users can access personalized features (tasks, habits, etc.).
Source	Module 1 – User Authentication
Rationale	Ensure data privacy and secure access for each user.
Business Rule	Each email must be unique; passwords must meet defined complexity standards.
Dependencies	None
Priority	High

3.3.2 Manage Tasks

In Table 3.3.2, the system shall allow users to create, edit, delete, and categorize tasks. Each task can include due dates, reminders, and recurring schedules. Users can view progress and manage tasks efficiently through a simple interface.

Table 3.3.2: Task Management

Field	FR-2
Title	Task Management
Requirement	• FR2.1: Users can create, edit, delete, and view tasks. • FR2.2: Users can set deadlines, and categories for tasks. • FR2.3: The system should send reminders or notifications for upcoming or overdue tasks. • FR2.4: Tasks can be marked as complete or incomplete. • FR2.5: Users can filter and sort tasks by date, priority, or completion status.
Source	Module 2 – Task Management
Rationale	Helps users organize and prioritize daily work effectively.
Business Rule	Each task must belong to one category; overdue tasks are highlighted automatically.
Dependencies	FR1 (Authenticate Users)
Priority	High

3.3.3 Track Habits

In Table 3.3.3, the system shall enable users to create and maintain daily, weekly, or monthly habits. It should record completions and display charts, calendars, and progress indicators.

Habit tracking helps users build consistency and monitor growth over time. The feature encourages discipline through visible progress. It supports long-term improvement goals.

Table 3.3.3: Habit Tracker

Field	FR3
Title	Habit Tracker
Requirement	• FR3.1: Users can add daily/weekly/monthly habits to track progress. • FR3.2: System should display visual progress (charts, streak lines, completion percentages), along with a calendar view of the habits. • FR3.3: Users can log habit completion each day. • FR3.4: The tracker should automatically update streaks when habits are maintained.
Source	Module – 3 Habit Tracker
Rationale	Encourages consistent personal growth and habit formation.
Business Rule	Streak resets if the habit is not completed on the specified day.
Dependencies	FR1 (Authenticate Users), FR4 (Calculate Streaks)
Priority	High

3.3.4 Calculate Streaks and Rewards

In Table 3.3.4, the system shall automatically update streak counters after successful completion of tasks and habits. It should track milestones, achievement levels, and badges to motivate users toward consistency. Missing scheduled habits or tasks may reset streak counts according to defined rules. Achievement levels will depend on the number of streaks maintained by the user.

Table 3.3.4: Streaks and rewards

Field	FR4
Title	Streaks and Rewards System
Requirement	• FR4.1: Completing daily habits increases user's streak count and progress level. • FR4.2: Streaks should be tracked when habits are completed continuously. Missing a scheduled activity should reset the streak. • FR4.3: Users shall unlock badges and achievement levels after reaching specific streak milestones. • FR4.4: The system should display streak history, milestones, and earned rewards on the dashboard.
Source	Module 4 – Streaks & Rewards System
Rationale	Motivates users to remain consistent through gamified streaks badges, and achievement levels.
Business Rule	Streaks update automatically after each logged completion. Badge levels depend on the number of streak milestones achieved.
Dependencies	FR1 (Authenticate Users), FR3 (Track Habits)
Priority	Medium

3.3.5 Give AI Recommendations

In Table 3.3.5, the system shall analyze user behavior and provide personalized recommendations. Suggestions may include better schedules, helpful content, learning resources, or productivity tips. AI responses should improve as more data becomes available. This module adds smart assistance and personalization. It helps users work more efficiently.

Table 3.3.5: AI recommendation

Field	FR5
Title	AI Recommendation
Requirement	- FR5.1: The AI should suggest helping material related to the user's tasks, such as YouTube videos or courses. - FR5.2: It should suggest things related to their added hobbies (e.g., new book recommendation) - FR5.3: It should suggest other related things to do based on their added hobbies.
Source	Module 5 – AI Recommendation
Rationale	Provides personalized learning and productivity guidance.
Business Rule	AI suggestions depend on the stored user habits and task keywords.
Dependencies	FR1 (Authenticate Users), FR2 (Manage Tasks), FR3 (Track Habits)
Priority	Medium

3.3.6 Allow Focus Tracking with Timer

In Table 3.3.6, the system shall provide a timer for focused work sessions and breaks. It must record completed sessions and generate productivity reports. Users can customize focus durations according to preference. This feature supports concentration and reduces distractions. It promotes structured work habits.

Table 3.3.6: Focus Tracker and Timer

Field	FR6
Title	Focus Tracker and Timer
Requirement	FR6.1: Users can start, pause, and reset a focus timer.

	• FR6.2: The system should record focus session durations and completed cycles. • FR6.3: Users can configure custom focus and break durations. • FR6.4: The system can generate weekly reports of total focused time.
Source	Module 7 – Focus Tracker and Timer
Rationale	Improves the time management and discipline through focused work sessions.
Business Rule	Timer sessions cannot exceed 120 minutes.
Dependencies	FR1 (Authenticate Users), FR2 (Manage Tasks)
Priority	Medium

3.3.7 Allow Note-Taking

In Table 3.3.7, the system lets users create, edit, and delete notes. It includes text, checklists, and tags for better searchability. Notes will be saved and synchronized with user account.

Table 3.3.7: Note-Taking

Field	FR7
Title	Note-Taking
Requirement	• FR7.1: Users can create, edit, delete, and organize notes. • FR7.2: Notes can be linked to tasks or habits. • FR7.3: Notes can include text, checklists, and tags for searching.
Source	Module 8 – Note-Taking
Rationale	Enables users to store ideas and maintain task-related information in one place.

Business Rule	Each note must have a unique title per account.
Dependencies	FR1 (Authenticate Users), FR2 (Manage Tasks)
Priority	Medium

3.3.8 Allow Collaboration with Other Users

In Table 3.3.8, the system shall enable users to communicate, share task lists, and collaborate on group projects within the app. Collaboration increases accountability and teamwork. It expands PlanLock from a personal tool into a social productivity platform.

Table 3.3.8: Community and Collaboration

Field	FR8
Title	Community and Collaboration
Requirement	• FR8.1: Users can join communities or groups with shared goals. • FR8.2: Users can post updates, comment, and react to others' posts. • FR8.3: Collaboration tools allow sharing of tasks or progress with teammates. • FR8.4: The system must handle content moderation (reporting and blocking).
Source	Module 9 – Community & Collaboration
Rationale	Promotes teamwork and accountability.
Business Rule	Messages and group activities must comply with user privacy settings.
Dependencies	FR1 (Authenticate Users)
Priority	Medium

3.3.9 Allow Theme Controls

In Table 3.3.9, the system shall enable users to personalize the interface through themes and accessibility settings. Users may choose light mode, and dark mode. Preferences should sync across devices automatically. Personalization improves comfort and accessibility. It creates a better overall user experience.

Table 3.3.9: Theme Control

Field	FR9
Title	Theme Control
Requirement	• FR11.1: Users can switch between multiple app themes (e.g., light mode, dark mode, system default). • FR11.2: The interface should automatically adapt to user-selected themes across all modules (tasks, notes, habits, etc.). • FR11.3: Accessibility settings (e.g., font size, color contrast) must be customizable.
Source	Module 12 – UI Personalization and Theme Control
Rationale	Improves accessibility and creates a personalized experience.
Business Rule	Theme preferences are stored per user and synced across device.
Dependencies	FR1 (Authenticate Users)
Priority	Low

3.4 Non-Functional Requirements

This section specifies the non-functional (quality) requirements for PlanLock – Personal Organizer mobile Application. These requirements ensure that the application not only meets functional requirements but also maintains high usability, reliability, security and scalability standards.

3.4.1 Usability

Usability requirements define how easily users can learn and interact with the system. Since PlanLock is targeted at students and professionals, it emphasizes a clean, intuitive, and accessible design to reduce cognitive load and enhance engagement. [6]

- **USE-1:** The system shall provide an intuitive and minimalist user interface that enables new users to understand and navigate major features (Task Management, Habit Tracker, AI Recommendation) within 5 minutes of first use.
- **USE-2:** The system shall allow users to perform primary actions (add a task, mark a habit, start a timer) within three taps or less.
- **USE-3:** The system shall display clear error messages for invalid inputs, ensuring users can recover from mistakes without external help.
- **USE-4:** The system shall support accessibility features, including adjustable font sizes, colour themes, and notification tones, to cater to diverse user preferences.

3.4.2 Performance

Performance requirements specify how well the system operates at both peak and typical usage levels. PlanLock's responsive design guarantees low latency and seamless synchronization.

- **PER-1:** With a steady 10 Mbps internet connection, the system should load the main dashboard in 3 seconds.
- **PER-2:** In less than two seconds following an update, the system will synchronize user data (tasks, habits, and streaks) with Firebase.
- **PER-3:** Without the need for a manual refresh, the system will show task and habit updates in real time.
- **PER-4:** Within five seconds of the appointed time, the user's device must receive notifications and reminders.
- **PER-5:** Up to 500 active user tasks and 200 habits per account must be handled by the system while maintaining steady performance.

3.4.3 Reliability & Availability

While availability determines how easily a system may be accessed by users, reliability guarantees that the system will continue to operate as intended over time.

- **REL-1:** When the system is operating normally, excluding maintenance intervals, it must have an uptime of at least 99%.
- **REL-2:** In the event of a brief network outage, the system will instantly reconnect to Firebase, restoring synchronization without erasing data.
- **REL-3:** To guarantee recovery in the event of a device loss, all task and habit data must be automatically backed up to the cloud once every 24 hours.
- **REL-4:** Users can view and update basic task and habit data, which will sync once the connection is restored. The system will function properly even if it is offline.
- **REL-5:** Upon restarting, the system will restore the most recent saved state in the event of an app crash or interruption.

3.4.4 Security

Security criteria specify how user information and system access are safeguarded. PlanLock places a high priority on data confidentiality, secure authentication, and user privacy.

- **SEC-1:** Through a secure email-password login, the system will employ Firebase Authentication to confirm user identity.
- **SEC-2:** Passwords must never be sent in plain text and must always be saved in hashed form.
- **SEC-3:** By blocking any unwanted access to analytics or personal data, the app will adhere to data protection regulations.

3.4.5 Scalability

Requirements for scalability specify how quickly the system can expand to accommodate higher user and data loads without seeing a decline in performance.

- **SCA-1:** The system architecture must be scalable to 10,000 users at once without compromising response times longer than three seconds.
- **SCA-2:** As the user base grows, the Firebase backend will enable dynamic expansion of bandwidth and data storage.

- **SCA-3:** New modules (such a web version or advanced AI integration) can be added to the program without requiring significant code reorganization thanks to its modular design.
- **SCA-4:** In the event that future improvements result in a 200% increase in data volume, the system will continue to operate steadily.
- **SCA-5:** To guarantee scalability and data integrity, the database structure must allow multi-user setups with separated storage for each account.

Chapter 4

DESIGN AND ARCHITECTURE

4. Design and Architecture

PlanLock is a mobile-based personal organizer application designed to consolidate planning, task management, habit tracking, note-taking, and collaboration into a single platform. It allows users to create, edit, categorize, and prioritize tasks with due dates, reminders, and difficulty levels. Habit tracking helps users monitor daily, weekly, or monthly routines, providing visual streaks, XP points, achievement badges, and notifications to encourage consistency. Additional modules include a focus timer for timed work sessions, AI-based recommendations for personalized guidance, note-taking with offline access, community collaboration with messaging and shared projects.

PlanLock is implemented for Android devices using Flutter for front-end development, Firebase for cloud storage. User authentication, privacy settings, and cloud synchronization ensure secure access and flexible control over personal and professional dashboards. The system emphasizes modularity, enabling scalability for future features like real-time collaboration and advanced AI integrations. By combining multiple functionalities in one app, PlanLock minimizes the stress of juggling multiple tools while promoting productivity and engagement.

4.1 System Architecture

System architecture defines the overall structure of PlanLock and explains how different components interact with each other. It provides a clear view of software modules, data flow, communication channels, and technology layers used in the system. A well-designed architecture ensures scalability, maintainability, performance, and security. PlanLock consists of a Flutter-based mobile frontend, cloud backend services, authentication modules, AI services, and notification systems. The frontend handles user interaction, while the backend manages business logic, synchronization, analytics, and data processing. Firebase is used for cloud data storage and real-time synchronization. This layered architecture improves flexibility and allows future feature expansion.

4.1.1 Architectural Design

In Figure 4.1.1, the system follows a Layered Architecture that separates the application into three main layers: the UI Layer, the Service (Logic) Layer, and the Database Layer. The UI

Layer handles all user interaction through screens such as task creation, login, and category management, ensuring that presentation remains independent from logic. It forwards all functional requests to the Service Layer, keeping the interface simple and focused on displaying information.

The Service Layer contains the core logic, processing user actions and managing communication with the Database Layer. Firebase Authentication handles secure sign-in, while Firestore stores data such as tasks and categories in structured collections. This modular separation ensures easier maintenance, scalability, and clear coordination between layers. The layers interact in a clean flow where the user interface requests logic, logic processes and accesses data, and results are returned to the user interface, creating a stable and extensible system architecture. [7]

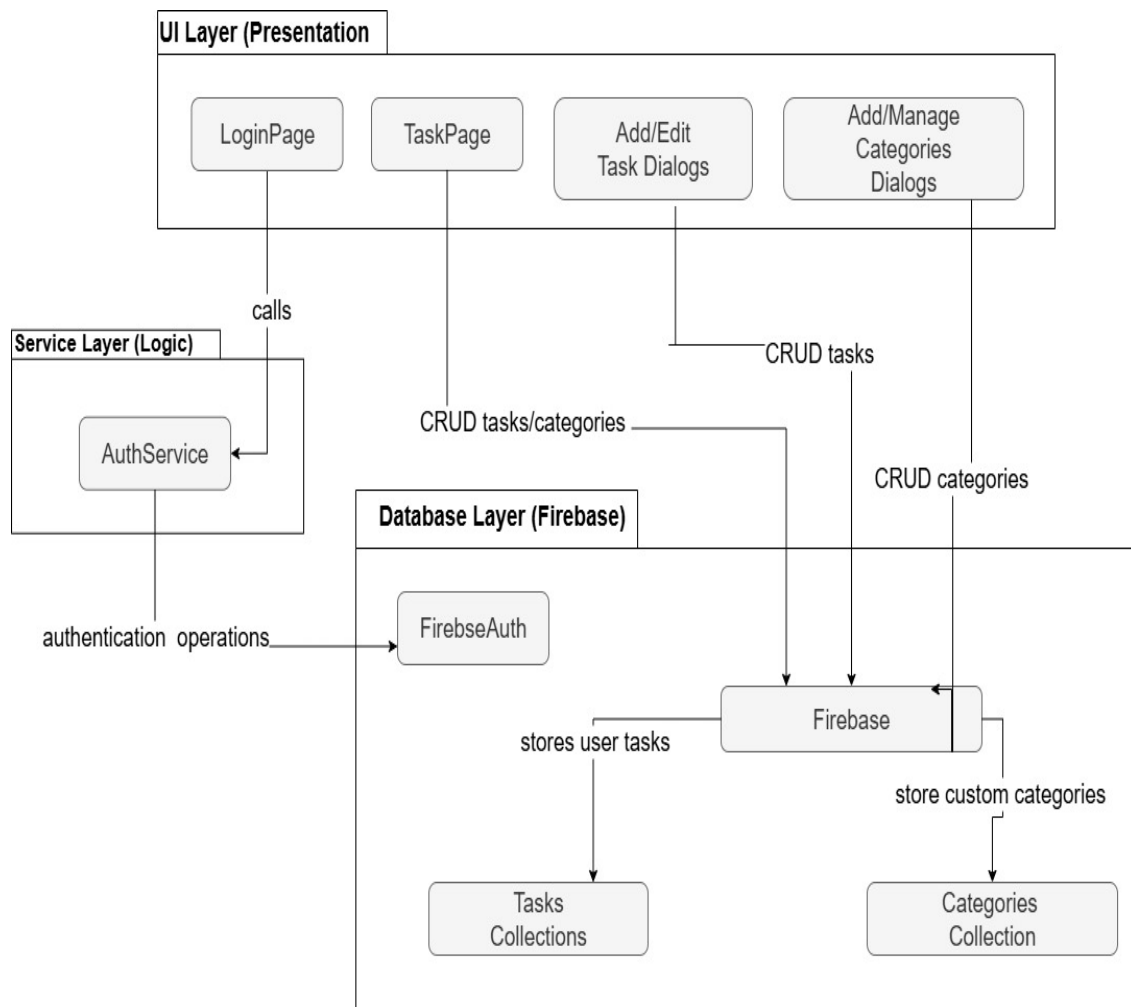


Figure 4.1.1: PlanLock Architecture Diagram

4.2 Data Representation

In PlanLock, the information domain is transformed into structured data entities that represent the core components of the system, such as users, tasks, habits, notes, and community. Each entity is modeled as an object with attributes (data fields) and methods (functions) that handle processing and updates.

Storage and Organization:

- **Firebase Cloud Firestore:** Used to store all user-related data, tasks, habits, notes, and gamification details in real-time. This ensures that updates are synchronized across devices instantly.

Data Processing:

- Operations like creating tasks, updating habits, logging streaks, and awarding points or badges are handled through the system's backend functions.
- The backend ensures data consistency, applies business rules (like streak calculation), and updates both local and cloud storage as needed.

Major Entities:

- **User:** Stores account and progress details.
- **Task:** Stores task information and tracks completion.
- **Habit:** Tracks daily, weekly, or monthly habit completion.
- **Note:** Stores personal notes with tags for searchability.
- **Reward:** Tracks points, badges, and streaks for gamification.

This design ensures that all system data is organized, accessible, and consistently updated, supporting both offline and online functionality.

4.2.1 Data Dictionary

The Data Dictionary of the PlanLock system defines the main data entities, attributes, and data types used in the PlanLock system. It provides a structured description of how data is stored

and managed within the application. The system uses Firebase for cloud storage. In Table 4.2.1, represents core module of the application including users, tasks, habits, notes, focus sessions, community interaction, AI recommendations, and user progress tracking.

Table 4.2.1: PlanLock Data Dictionary

Entity	Attribute Name	Data Type	Constraints	Description
Users	user_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique identifier for each user (Firebase UID).
	email	VARCHAR (255)	UNIQUE, NOT NULL	User's email address.
	password_hash	VARCHAR (255)	NOT NULL	Hashed and salted password.
	username	VARCHAR (100)	UNIQUE, NOT NULL	Display name for the user.
	profile_picture	TEXT	NULL	URL or path to profile image.
	. bio	TEXT	NULL	Short biography.
Tasks	task_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique task identifier.
	user_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	Owner of the task.
	title	VARCHAR (255)	NOT NULL	Task title.

	description	TEXT	NULL	Detailed task description.
	category	ENUM('Daily', 'Weekly', 'Monthly')	NOT NULL	Task category.
	priority	ENUM('Low', 'Medium', 'High')	DEFAULT 'Medium'	Task priority.
	Due_date	DATETIME	NULL	Task deadline.
	is_completed	BOOLEAN	DEFAULT FALSE	Completion status.
Habits	habit_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique habit identifier.
	user_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	Owner of the habit.
	habit_name	VARCHAR (255)	NOT NULL	Name of the habit.
	frequency	ENUM('Daily', 'Weekly', 'Monthly')	NOT NULL	Tracking frequency.
	current_streak	INT	DEFAULT 0	Current consecutive

				days completed.
	longest_streak	INT	DEFAULT 0	Longest streak achieved.
Notes	Note_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique note identifier.
	User_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	Owner of the note.
	title	VARCHAR (255)	NOT NULL	Note title.
	content	INT	NULL	Note body content.
	tags	JSON	NULL	Array of tags for categorization.
Focus_Sessions	session_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique session identifier.
	user_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	User who started the session.
	start_time	TIMESTAMP	NOT NULL	Session start time.
	end_time	TIMESTAMP	NULL	Session end time.
	duration_minutes	INT	NULL	Total focused minutes.

	session_type	ENUM('Focus', 'Break')	DEFAULT 'Focus'	Type of session
Community_Posts	post_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique post identifier.
	user_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	Author of the post.
	content		NOT NULL	Post content.
AI_Recommendations	recommendation_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique recommendation identifier.
	user_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	User receiving the recommendation.
	content_type	ENUM('Article', 'Video', 'Course', 'Book', 'Tool')	NOT NULL	Type of recommended content.
	title	VARCHAR (255)	NOT NULL	Title of the recommendation.
	description	TEXT	NULL	Description of the recommendation.

	url	TEXT	NULL	Link to the recommended resource.
	category	VARCHAR (100)	NULL	Category (e.g., Productivity, Learning, Health).
User_Progress	progress_id	VARCHAR (50)	PRIMARY KEY, NOT NULL	Unique progress record ID.
	user_id	VARCHAR (50)	FOREIGN KEY (Users.user_id)	Reference to the user.
	total_xp	INT	DEFAULT 0	Total experience points earned.
	current_level	INT	DEFAULT 1	Current user level.
	last_activity_date	DATE	NULL	Last date the user was active.
	daily_streak	INT	DEFAULT 0	Current daily login streak.

In Figure 4.2.1, the ER diagram of PlanLock illustrates the structural relationships among the main entities of the system, including User, Tasks, Habits, Notes, Focus Sessions, Community, AI Recommendations, and User Progress. It provides a clear and organized view of how data is stored, managed, and interconnected across different modules of the application. Each entity represents a core functional component of the system, ensuring that all user activities are systematically recorded and maintained in a structured database design.

The User entity serves as the central entity of the system, where each user is linked to multiple related entities such as tasks, habits, notes, focus sessions, and progress records. This establishes a one-to-many relationship between the User and these entities, meaning that a single user can create, update, and manage multiple records in each module. The diagram also highlights how user progress data is continuously updated based on daily activities, enabling tracking of consistency, performance, and engagement over time.

Furthermore, the ER diagram shows the relationship between users and the Community module, where users can participate in posts, discussions, and collaborative activities. It also demonstrates how AI Recommendations are connected with user behavior data to generate personalized suggestions and responses. Overall, the ER diagram ensures proper relationship mapping, data consistency, and efficient organization of the PlanLock system, supporting smooth functionality across all modules.

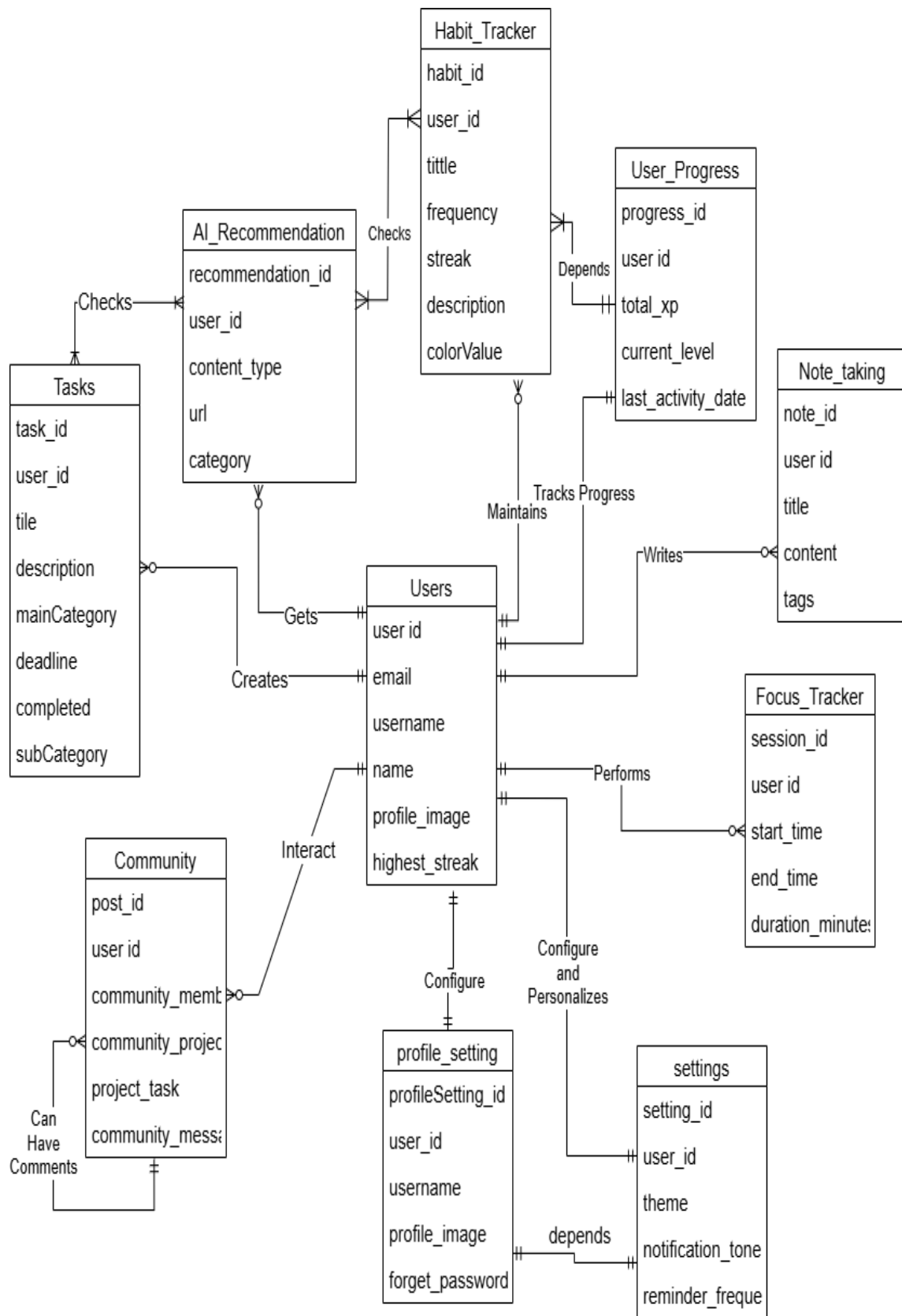


Figure 4.2.1: PlanLock Entity Relationship Diagram

4.3 Process Flow Representation

This section describes the major processes within the PlanLock system and their sequence of operations. The focus is on key workflows critical to the platform's functionality.

User Registration and Login

- Users access the registration or login page.
- New users provide their personal details and create a secure account. Existing users enter their credentials to log in.
- The system verifies the credentials and redirects users to the main dashboard upon successful authentication.

Task Management

- Users navigate to the Create Task page.
- They enter task details such as title, description, category, difficulty level, deadline, and reminders.
- Upon submission, the task is saved in the database, displayed in the dashboard, and made available for progress tracking and notifications.

Habit Tracking

- Users create or update habits with daily, weekly, or monthly schedules.
- The system monitors completion, updates streaks, points, and achievement badges, and sends reminders for missed days.
- Analytics and progress graphs provide insights into consistency and habit-building trends.

AI Recommendations

- The AI module analyzes the user's habits.
- Personalized suggestions are provided to help users improve consistency and habit-building.
- Recommendations are updated dynamically as user habit data evolves.

Focus Timer

- Users start a timer for any activity they choose.
- The system records the time spent during the session.
- Time spent are recorded, and monthly charts of activity are generated for review.

Note-Taking

- Users create notes via typing or voice input.
- Notes are organized using tags and made searchable for easy retrieval.
- Offline access is supported, while shared notes in the community are synced when online.

Community & Collaboration

- Users can send direct messages, create projects, join groups, share tasks, and view profiles within the community.
- Progress and deadlines of group tasks are tracked collectively to encourage teamwork.
- Privacy settings allow users to control visibility of streaks, XP, and achievements

4.4 Design Model

The design model defines the structural and behavioral view of the PlanLock system using UML diagrams. It explains how different components of the system interact with each other and how the system is organized internally. These models help in understanding the system design clearly before implementation and ensure proper communication between modules.

4.4.1 Class Diagram

In Figure 4.4.1, the User is the central class, owning Tasks, HabitTracker entries, Notes, FocusSessions, and UserProgress, ensuring nothing can exist without a logged-in user. AIRecommendation depends on HabitTracker to provide personalized suggestions, while UserProgress is updated based on habit completion. Communities exist independently and associate with Users, allowing interaction without ownership.

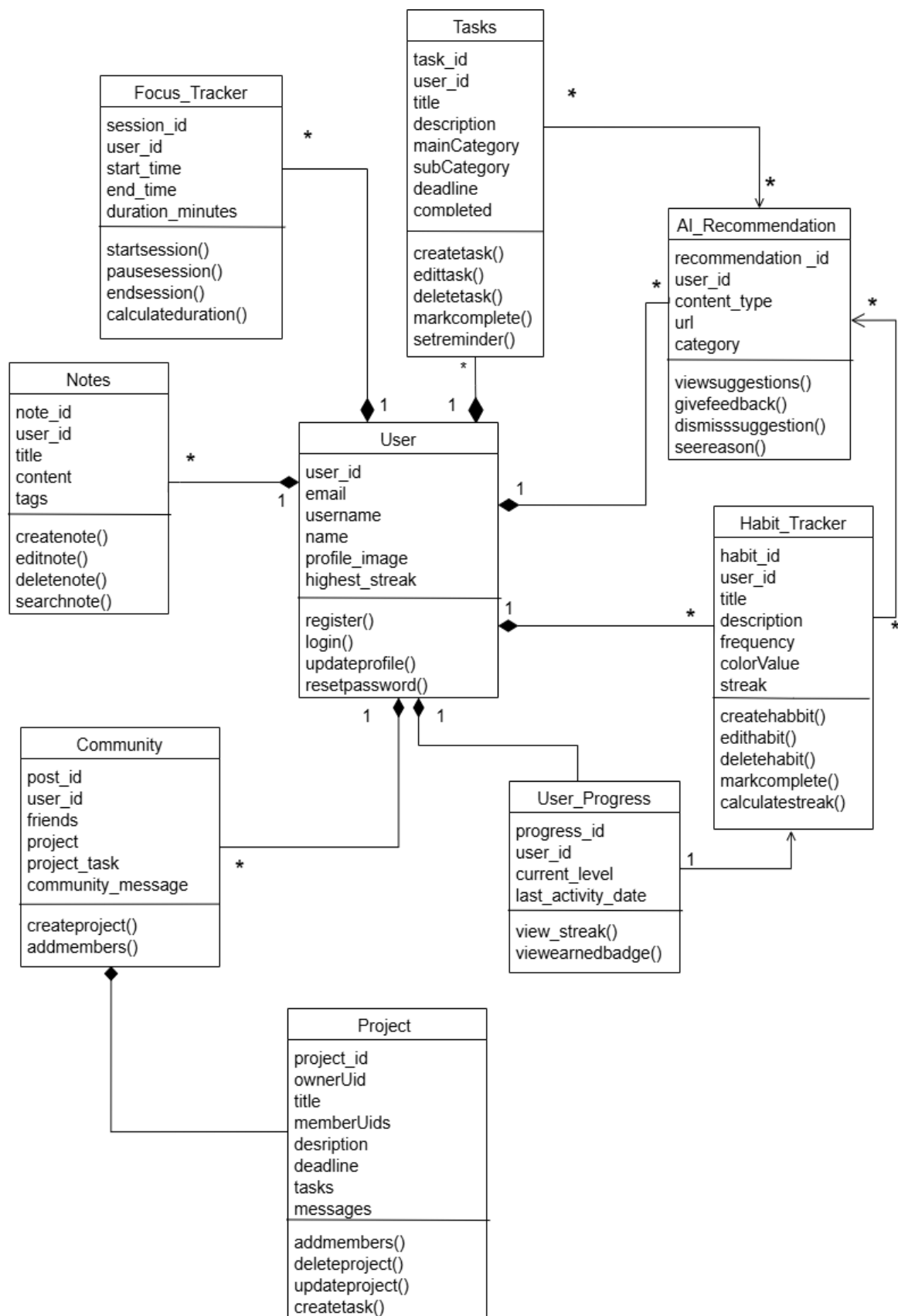


Figure 4.4.1: PlanLock Class Diagram

4.4.2 Sequence Diagram

In Figure 4.4.2, the sequence diagram shows the step-by-step interaction between the user, mobile application, system modules, and database during a process. It explains how requests such as login, adding tasks, tracking habits, or starting a focus session are sent from the user interface to the backend and then stored or retrieved from the database.

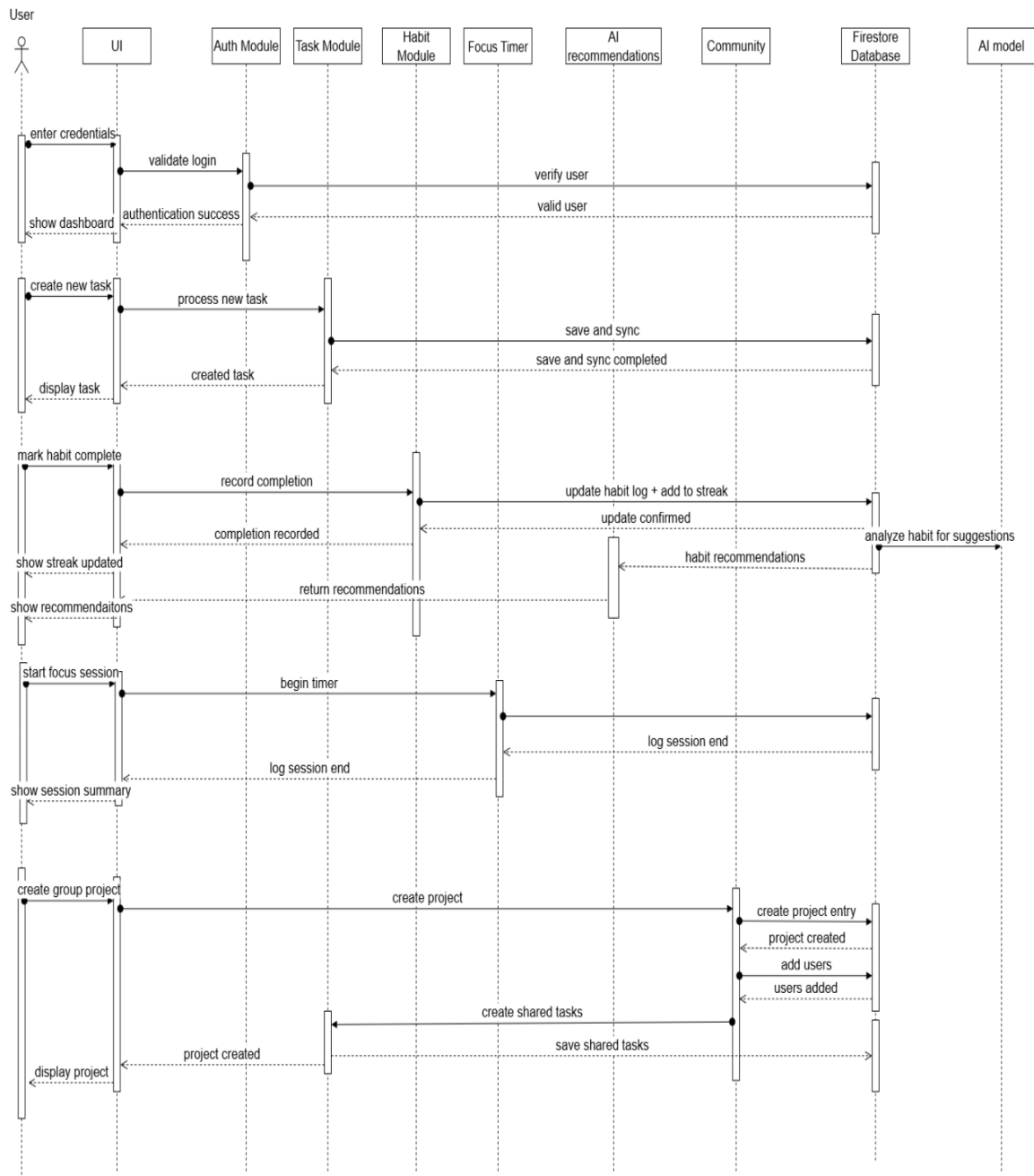


Figure 4.4.2: PlanLock Sequence Diagram

4.4.3 State Transition Diagram

In Figure 4.4.3, the state transition diagram illustrates the different states of the system or an object and how it moves from one state to another based on user actions or events. For example, a task can move from Created to Pending, In Progress, Completed, or Overdue states. It helps visualize the lifecycle of tasks, habits, or user sessions within the application. This diagram is useful for understanding system behavior dynamically.

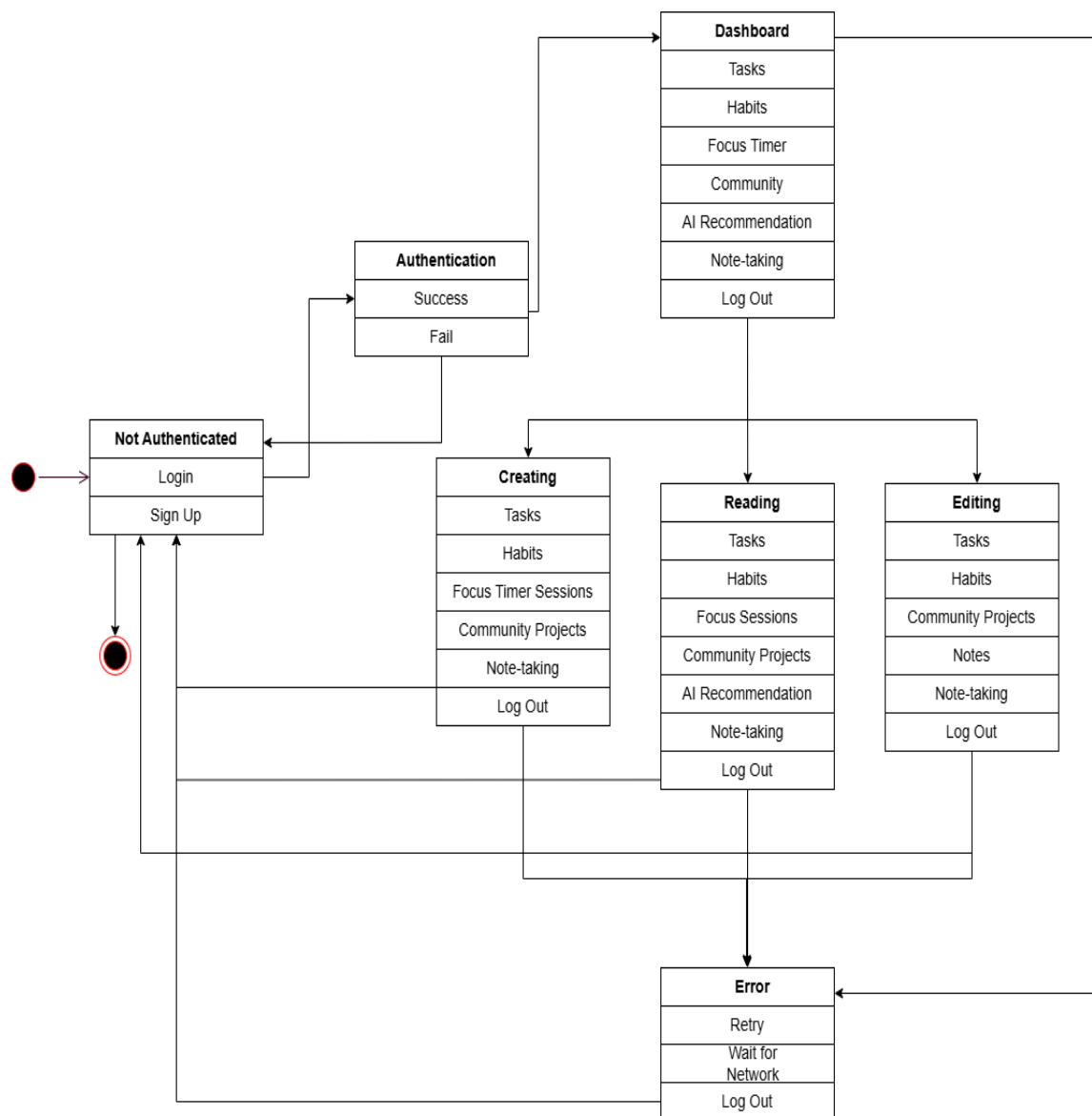


Figure 4.4.3: PlanLock State Transition Diagram

4.4.4 Data Flow Diagram

Figure 4.4.4-1, Level 0 Data Flow Diagram gives a high-level overview of the complete system. It shows the main processes such as User Authentication, Task Management, Habit Tracking, Focus Timer, AI Recommendation, Notes, and Community Collaboration. It also represents the interaction between external users and the main system databases.

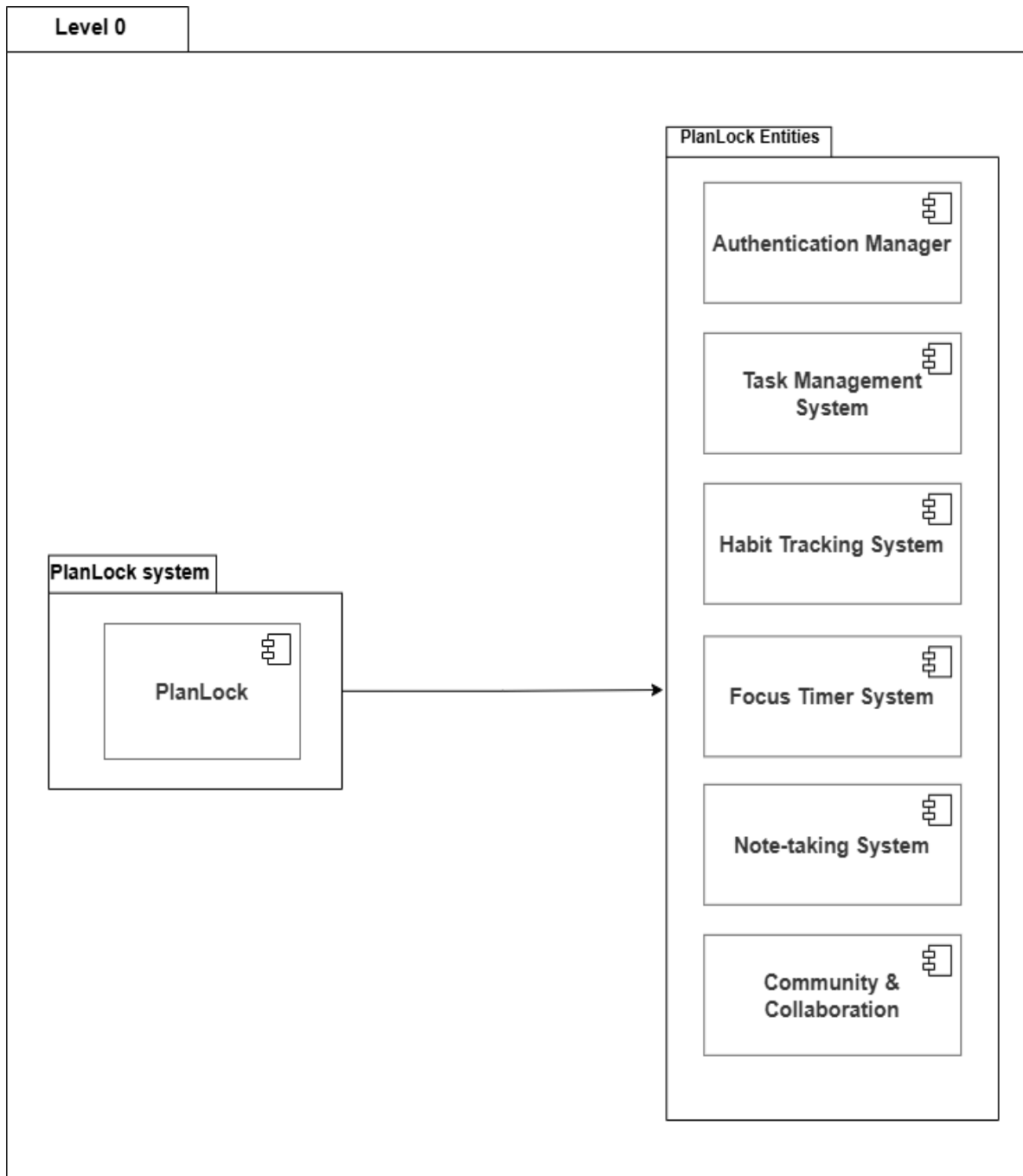


Figure 0.4-1: PlanLock Data Flow Diagram (Level 0)

In Figure 4.4.4-2, Level 1 Data Flow Diagram provides a detailed breakdown of the major processes shown in Level 0. It explains sub-processes such as user registration, login validation, creating tasks, editing tasks, marking habits complete, updating streaks, saving notes, and generating reports. It shows more specific data movement between modules and storage.

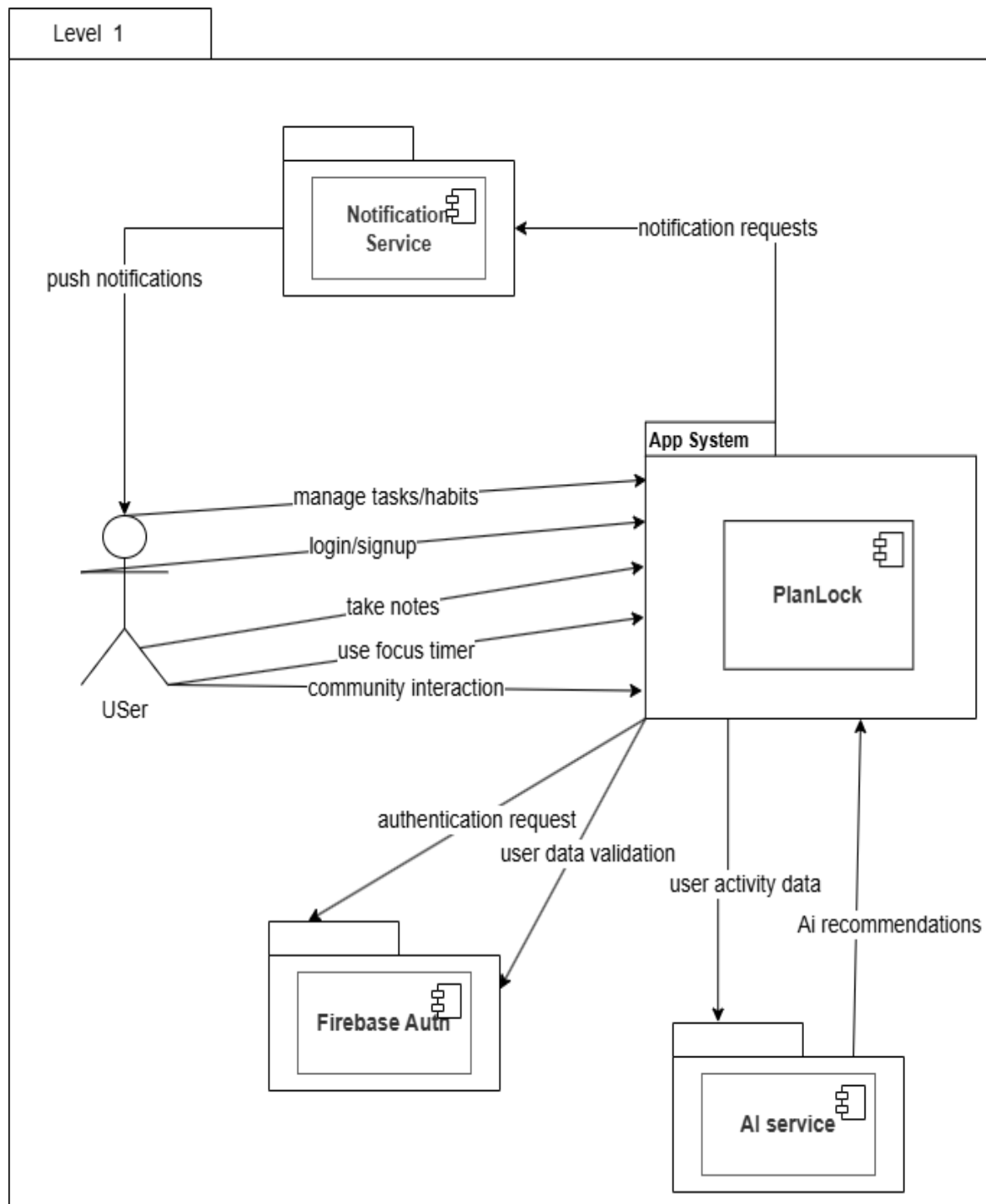


Figure 0.4-2. PlanLock Data Flow Diagram (Level 1)

In Figure 4.4.4-3, Level 2 Data Flow Diagram gives a very detailed internal view of a selected sub-process from Level 1. It describes step-by-step operations such as validating login credentials, generating AI suggestions, or syncing offline data with Firebase. This level helps in understanding the internal logic of the system clearly.

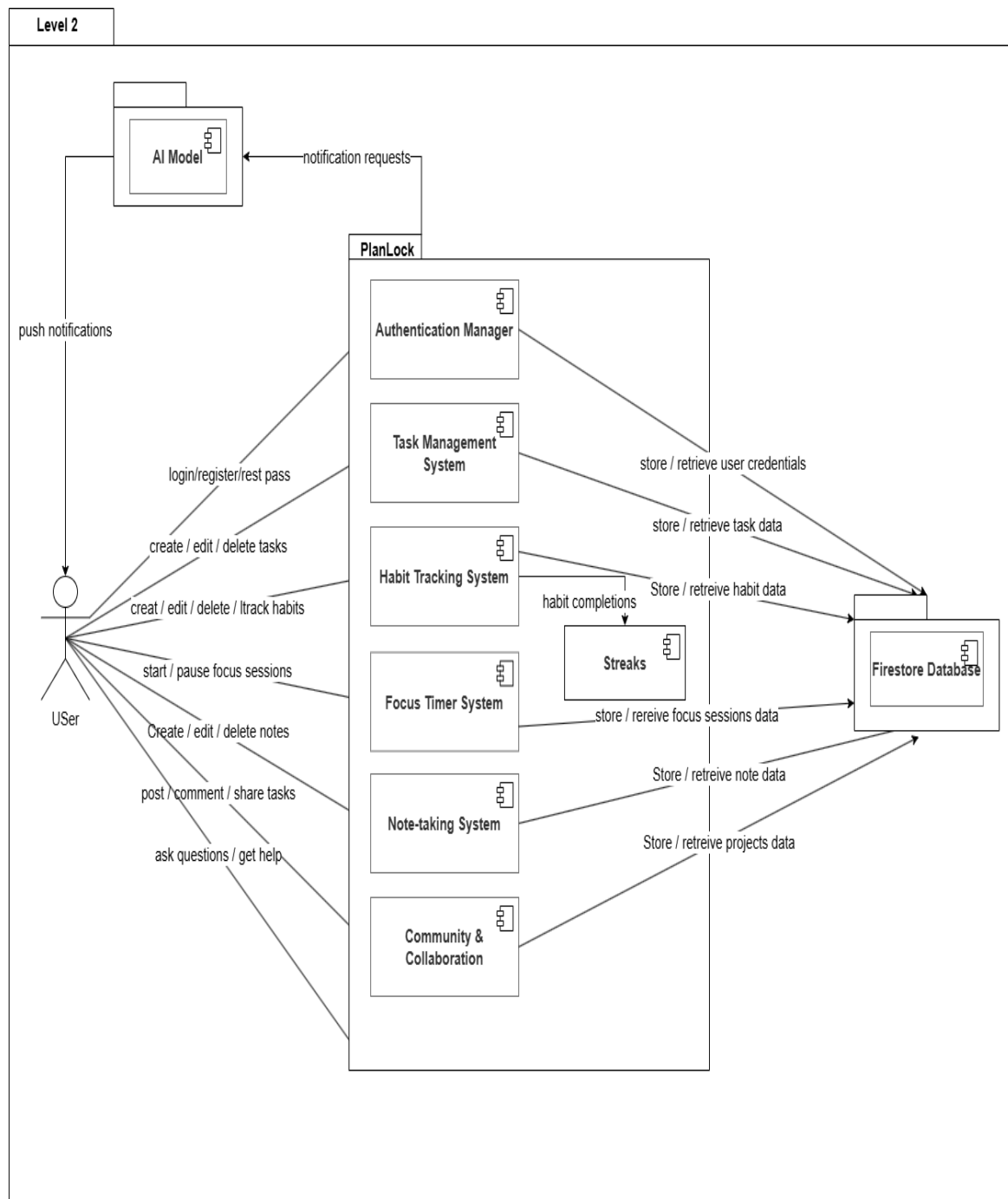


Figure 0.4-3. PlanLock Data Flow Diagram (Level 2)

In Figure 4.4.4-4, Level 3 Data Flow Diagram provides the most detailed view of the system by further decomposing a specific Level 2 process into smaller internal operations. It explains minute data movements such as field validation, error handling, database queries, notification triggering, session logging, and response generation. For example, in user login, it may show email verification, password hash matching, session creation, and dashboard loading. This level is useful for developers to understand implementation details and system logic precisely.

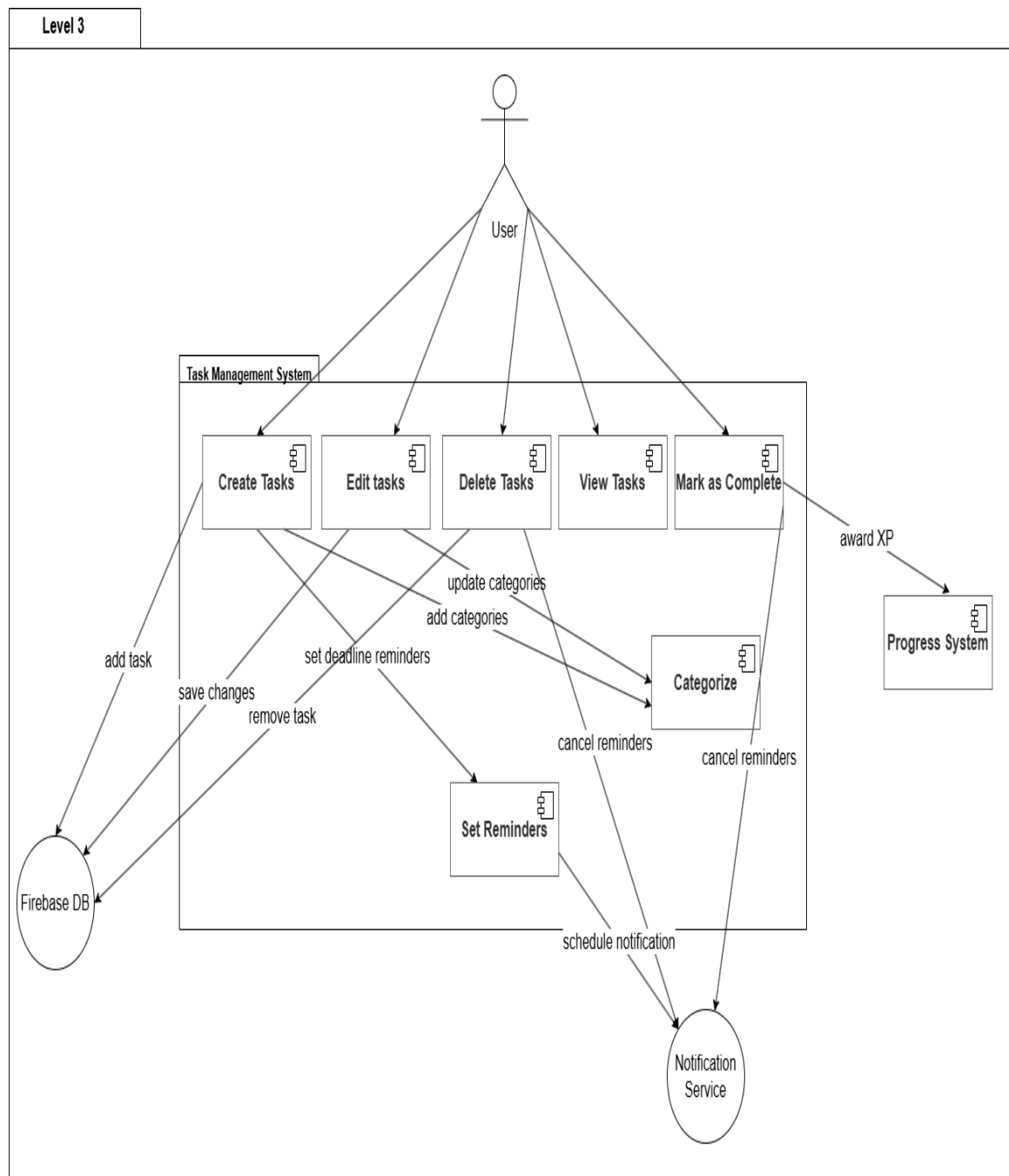


Figure0.4-4. PlanLock Data Flow Diagram (Level 3)

Chapter 5

IMPLEMENTATION

1. Implementation

This explains the implementation details of the PlanLock system. It describes how the system requirements and design models are converted into working features. The implementation focuses on core modules such as user authentication, task management, habit tracking, streak system, AI recommendations, and other productivity tools. It ensures that all functional requirements are properly developed and integrated into a unified system.

5.1 Algorithm

This section describes the main algorithms used in the PlanLock system to implement its core functionalities. The algorithms define step by step procedures for handling user actions and system responses.

5.1.1 User Authentication: This algorithm defines the secure process for user registration, login, and password recovery in the PlanLock system to ensure authenticated access to the application.

Steps:

1. Open the PlanLock app and tap "Sign Up" or "Login."
2. Select authentication method: email/password or social login.
3. Enter required credentials (email, password) and submit.
4. If credentials are invalid → view error message and retry.
5. If password is forgotten → tap "Forgot Password" → enter email → reset via OTP or link.
6. On successful validation → get redirected to the home dashboard.
7. If authentication service is unavailable → wait for system retry notification.

5.1.2 Create and Manage Tasks: Explains how users create, manage, update, and track tasks in the PlanLock system, including scheduling, categorization, and completion handling to improve productivity and organization.

Steps:

1. Log in and navigate to the Tasks section.
2. Tap "Add Task" to create a new task.
3. Enter task title (mandatory), description, deadline, and select a category (Daily/Weekly/Monthly).
4. Tap "Save" → task appears in the task list.
5. To edit an existing task → select it → update fields → save changes.
6. To delete a task → select it → confirm deletion → task is removed.
7. Mark a completed task using the completion checkbox.
8. If a deadline is set → system schedules a notification automatically.
9. If save fails due to network error → tap "Retry" to save again.
10. View all active tasks organized by category in the task list dashboard.

5.1.3 Create and Track Habits: This algorithm describes how users create, manage, and track habits in the PlanLock system, enabling them to maintain routines, monitor consistency through progress visualization, and improve long-term discipline.

Steps:

1. Log in and navigate to the Habit Tracker section.
2. Tap "Add Habit" to create a new habit.
3. Enter habit name and select frequency (daily/weekly).
4. Tap "Confirm" → habit appears in the habit list.
5. Each day, mark habits as complete using the daily tracker.
6. To edit a habit → select it → update name or frequency → save.
7. To delete a habit → select it → confirm removal.

8. View visual progress charts to monitor consistency over time.
9. If habit save fails due to sync issue → system retries automatically.
10. Manage up to a maximum of 10 active habits simultaneously.

5.1.4 Earn Streaks and Rewards: This describes the gamification process in PlanLock where users maintain streaks and earn badges by consistently completing tasks and habits.

Steps:

1. Log in and ensure at least one active task or habit exists.
2. Complete a task → system increases streak count automatically.
3. Complete a habit → system updates daily streak progress.
4. System increments streak counter for each consecutive day of activity.
5. View current streak and earned badges via notification after each completion.
6. Collect badges and achievement milestones as streaks increase.
7. If no activity is recorded within 24 hours → streak resets to zero.
8. If streak breaks → system notifies user and resets streak count.
9. If offline → streak update is queued and synced on next connection.
10. Track total streak history and achievements on the Progress Dashboard.

5.1.5 AI Recommendations: This describes how PlanLock uses AI to analyze user activity and provide personalized recommendations to improve productivity, scheduling, and habits over time.

Steps:

1. Log in and ensure at least 7 days of activity data is recorded.
2. Navigate to the "AI Insights" tab.
3. System analyzes usage, focus patterns, and productivity behavior.
4. AI generates personalized suggestions for tasks, habits, and schedules.

5. View displayed recommendations on the Insights dashboard.
6. Apply or dismiss suggestions based on personal preference.
7. Tap "Refresh" to manually trigger AI to regenerate new insights.
8. If less than 7 days of data exists → view "Not enough data" message.
9. AI adapts and improves recommendations over time with more activity.
10. Receive daily automated analysis even without opening the Insights tab.

5.1.6 Focus Tracker with Timer: This describes how PlanLock's Focus Tracker helps users manage structured work sessions using a timer system to improve concentration and productivity through focused intervals and breaks.

Steps:

1. Log in and navigate to the Focus Tracker section.
2. Tap "Start Focus Session" to begin the timer.
3. System starts a 25-minute countdown (default duration).
4. Stay focused during the session — avoid interruptions.
5. At timer completion → receive break notification from the system.
6. Take a break, then start the next session when ready.
7. To pause or end early → tap pause/stop → partial session data is saved.
8. View logged focus session history and total focused time on dashboard.
9. If app crashes mid-session → system restores previous timer state on relaunch.
10. Complete at least one session daily to maintain streak.

5.1.7 Note-Taking: This describes how PlanLock allows users to create, organize, edit, and manage personal notes efficiently, with options for categorization, and secure private storage. It also helps users quickly capture ideas and retrieve important information whenever needed, improving overall productivity and organization.

Steps:

1. Log in and navigate to the Notes section.
2. Tap "New Note" → system opens the note editor.
3. Write note content including ideas, reflections, or reminders.
4. Categorize the note and optionally link it to a specific task.
5. Tap "Save" → note appears in the notes list.
6. To edit a note → select it → modify content → save changes.
7. To delete a note → select it → confirm removal.
8. Use the search bar to quickly find notes by keyword or category.
9. If save fails → tap "Retry" to attempt saving again.
10. All notes are private by default and only visible to the logged-in user.

5.1.8 Community & Collaboration: This describes how PlanLock enables users to interact, collaborate, and share progress through a community platform that supports posts, comments, group challenges, and real-time engagement for improved motivation and accountability.

Steps:

1. Log in and tap the "Community" tab.
2. Browse the community feed showing posts, goals, and group activities.
3. Tap "Create Post" → write content → submit to share with community.
4. Comment on others' posts to engage and offer motivation.
5. Join a group challenge to collaborate on shared productivity goals.
6. Track shared progress and team milestones within the group.
7. If inappropriate content is found → tap "Report" to flag it for moderation.
8. Receive notifications when others interact with your posts or comments.
9. If server times out → view error message and tap "Retry" to reload.
10. Ensure all posts comply with community guidelines (auto-moderated by system).

5.2 External APIs

5.2.1 Firebase Authentication API: This API handles all user identity and security features within the app. It powers the Login, Signup, and Reset Password screens by managing secure account creation, encrypted credential verification, and automated password recovery emails. Within the Profile and Settings modules, it ensures that only authorized users can access and modify personal data, while providing persistent session management so users stay logged in across app restarts.

5.2.2 Cloud Firestore API: Firestore acts as the primary real-time database API, facilitating the storage and synchronization of all user-generated content. In the Task Management and Habit Tracker modules, it handles the creation, updating, and deletion of tasks and routines with instant cloud sync. The Notes and Focus Tracker modules rely on it to save detailed logs and formatted text, while the Community module uses its real-time listeners to power group project collaboration and live chat messaging.

5.2.3 Google Generative AI (Groq API): Integrated via the `google_generative_ai` package. It uses advanced natural language processing to understand user habits provide helpful recommendation and productivity strategies along with links to helpful external sources.

5.2.4 Firebase Remote Config API: This service is used as a secure management layer for application parameters that need to be updated without a new app release. Its primary role in PlanLock is to securely store and deliver the Groq API Key, ensuring that sensitive credentials are not hardcoded into the app's source code. This API allows the developers to modify backend configurations or switch AI model versions dynamically across all user devices simultaneously.

5.2.5 Image Picker and Path Provider APIs: These local device APIs are utilized in the Profile and Settings modules to manage user avatars. The Image Picker API provides the interface for users to select images from their photo gallery or capture new ones using the device camera. The Path Provider API works alongside it on mobile platforms to identify secure local directory paths, allowing the app to save and retrieve profile pictures from the device's internal storage for offline access and faster rendering.

5.2.6 Shared Preferences API: This is a lightweight data persistence API used primarily in the Settings and App Layout modules to store local user preferences. It tracks the user's choice of Dark Mode or Light Mode and caches basic profile information like the user's name. By

storing these small key-value pairs locally on the device, the app can immediately apply the user's preferred visual theme and display their name on the Home Page before the faster Firestore network calls even complete.

5.3 User Interface

This presents the graphical user interface (GUI) of the PlanLock application. It includes the main screens of the system along with their descriptions and functionalities. Each screen is designed to provide a simple, user-friendly, and intuitive experience for users while interacting with different modules such as tasks, habits, focus timer, notes, AI recommendations, and community features. The interface ensures smooth navigation and consistency across all features of the application. [8]

- **Login screen (login.dart):** Figure 5.3.1, the Login Screen provides a secure authentication gateway for returning users to access their personal accounts. It includes distinct fields for email and password input alongside robust error validation logic. A convenient link to password recovery workflows is featured for users who have forgotten their credentials. Upon successful verification, it seamlessly transitions the user to the main home dashboard.

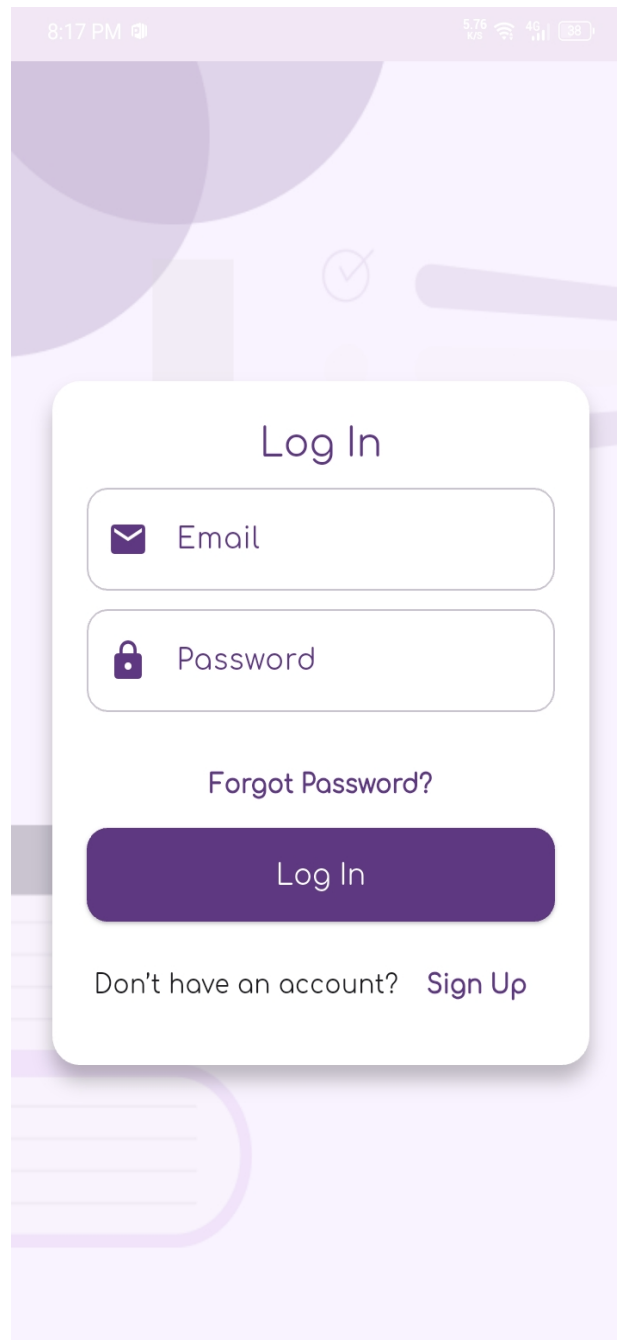


Figure 5.3.1: Login Screen

- **Sign Up Screen (signup.dart):** In Figure 5.3.2, the Sign Up Screen enables new users to securely register and create a profile within the PlanLock ecosystem. It captures essential user details such as an email address, a unique username, and a secure password. The screen integrates seamlessly with backend Firebase authentication to safely store user credentials. It guides users smoothly into the application's main interface immediately after the account is established.

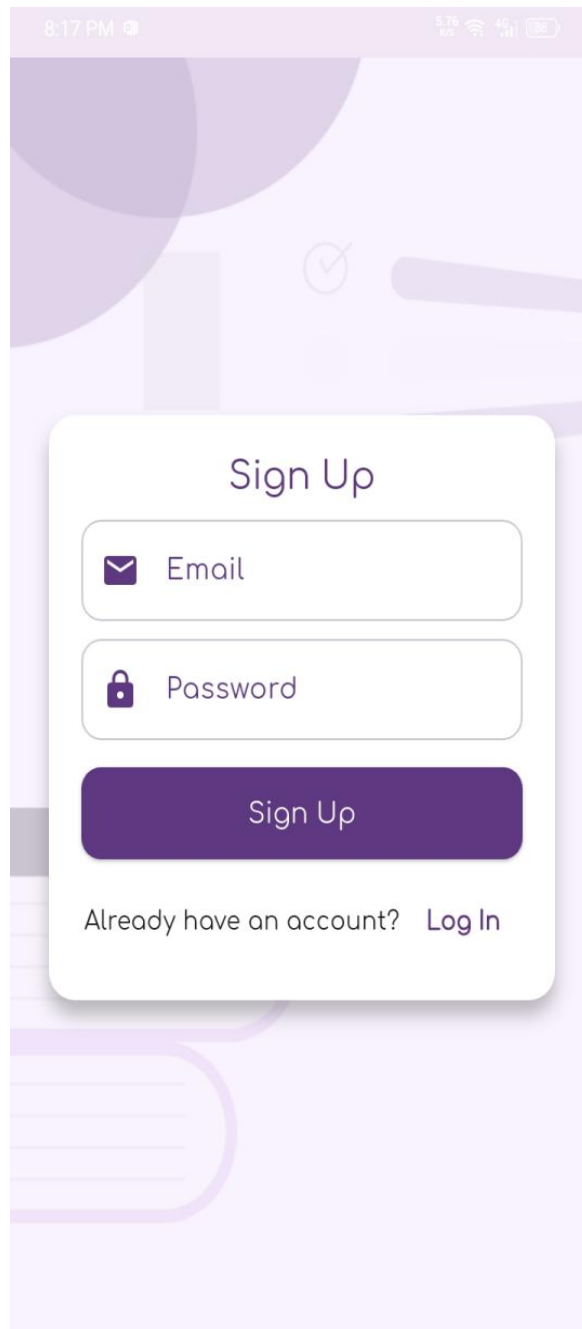


Figure 5.3.2: Signup Screen

- **Reset Password Screen (reset_password.dart):** In Figure 5.3.3, The Reset Password Screen offers a simple recovery mechanism for users who have temporarily lost access to their accounts. It prompts the user to enter their registered email address to receive official reset instructions. The screen connects to Firebase authentication to securely dispatch automated password reset emails. It maintains user engagement by ensuring the account recovery process is frictionless and straightforward.

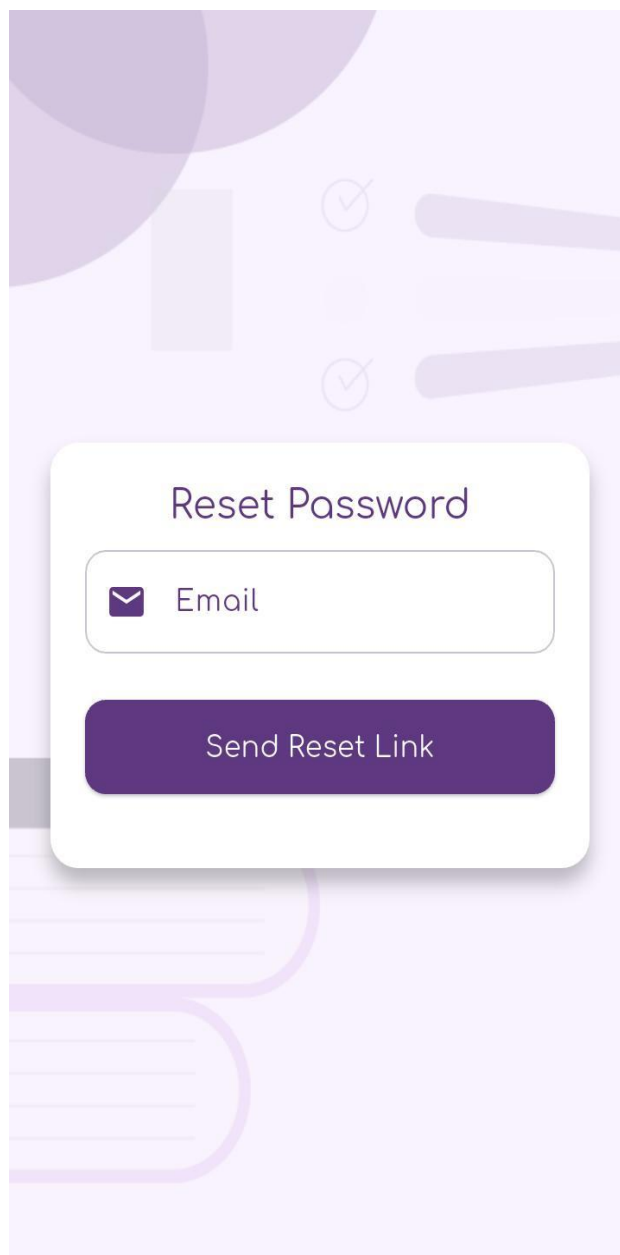


Figure 5.3.3. Rest Password Screen

- **Reset Password Email Notification:** In Figure 5.3.4, the Reset Password Email Notification confirms that Firebase Authentication has successfully dispatched an official password recovery message to the user's registered email address. The email is sent automatically upon submitting the Reset Password form and contains a secure, time-limited link generated by Firebase. Clicking the link redirects the user to a safe password update interface, completing the full account recovery workflow. This notification validates that the Flutter frontend and Firebase Authentication backend are correctly integrated end to end.

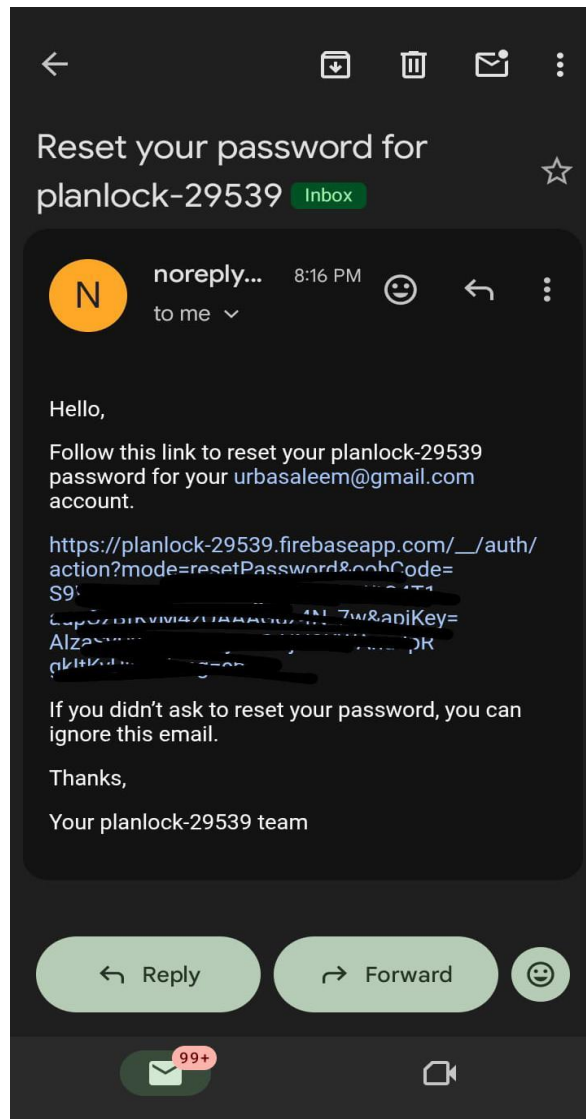


Figure 1.3.4: Reset Password Link Email

- **Profile Screen (profile_page.dart):** In Figure 5.3.5, the Profile Screen neatly displays the user's personal information, account details, and cumulative productivity statistics. It allows users to easily customize their avatar, update display names, and fully manage their identity. Major achievements and overall progress across habits, tasks, and focus timer sessions are summarized here. It provides a centralized, visually appealing location for users to review their journey and app usage history.

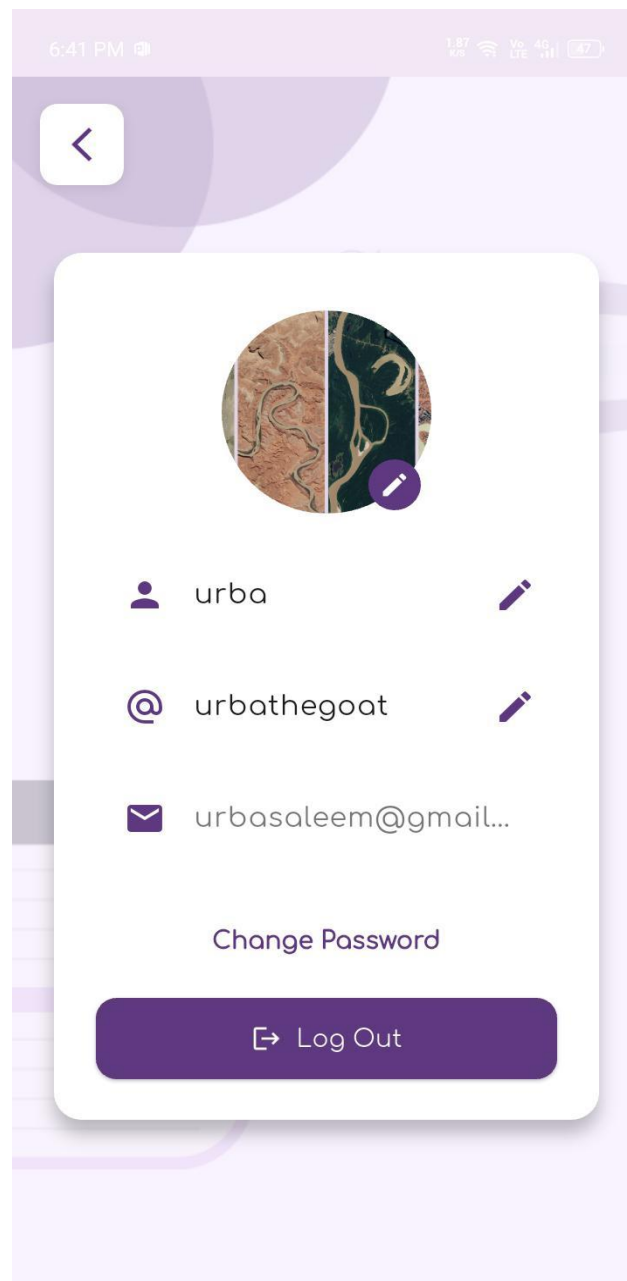


Figure 1.3.5: Profile Page

- **Edit Profile:** In Figure 5.3.6, this interface allows users to update their display name and profile picture through the settings or profile section. Mobile users can upload images from the gallery, while web users can upload pictures using base64 image support. It also includes a text field for editing the full name with an auto-save option linked to Firestore. All changes are synchronized instantly with the database and local storage.

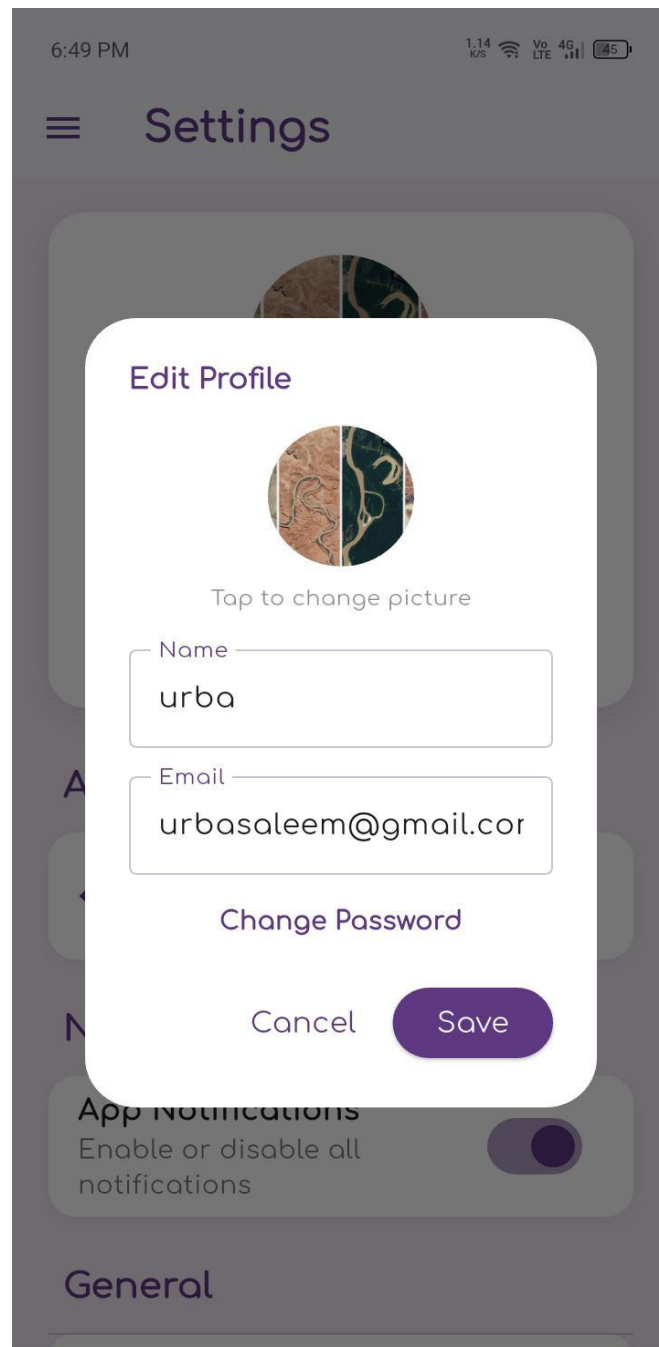


Figure 5.3.6: Edit Profile

- **Home Dashboard (home_page.dart):** In Figure 5.3.7, the Home Dashboard acts as the central command hub providing a high-level overview of the user's daily productivity. It displays quick summaries of active habits, upcoming tasks, and the latest unread notifications. The screen serves as the primary navigation anchor, linking out fluidly to all other major application modules. It features dynamic UI elements that instantly adapt to reflect the user's daily progress and state.

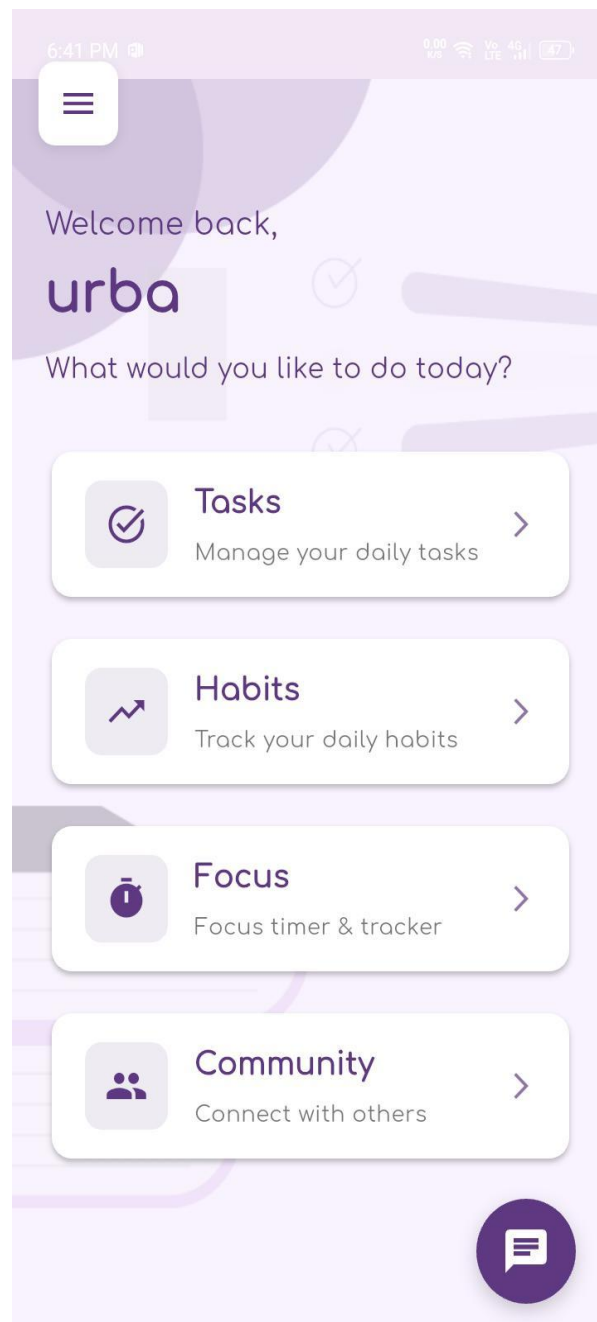


Figure 5.3.7: Home Page

- **Task Manager (task_page.dart):** In Figure 5.3.8, the Task Manager provides a comprehensive system for organizing, prioritizing, and completing daily to-do items. It supports adding strict deadlines, detailed descriptions, and categorizing tasks for optimal focus. An intuitive checklist interface allows users to easily mark items as finished with a tap. It helps maintain high productivity by ensuring no important assignment or chore slips through the cracks.

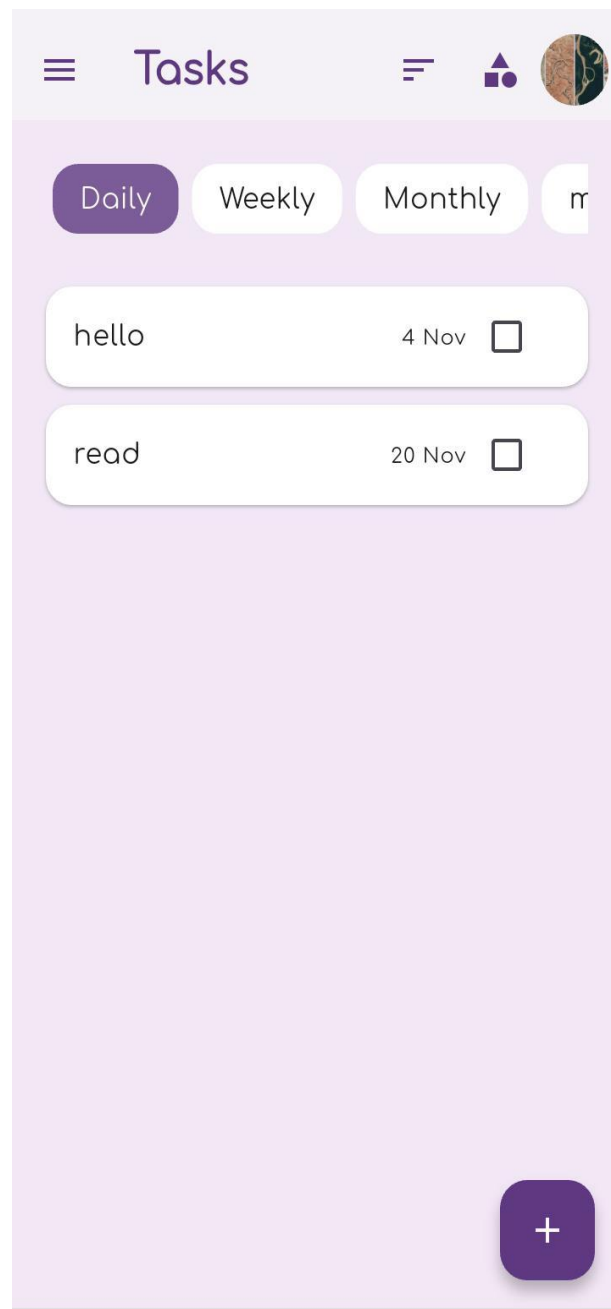


Figure 5.3.8: Task Page

- **Add Task:** In Figure 5.3.9, the Add Task popup interface is used for quickly adding new tasks into categorized task lists. It contains fields for task title, description, and a date picker for selecting deadlines. Users can assign tasks to categories such as Daily or Weekly, or create custom categories. The dialog follows the app theme and integrates smoothly with the task management workflow.

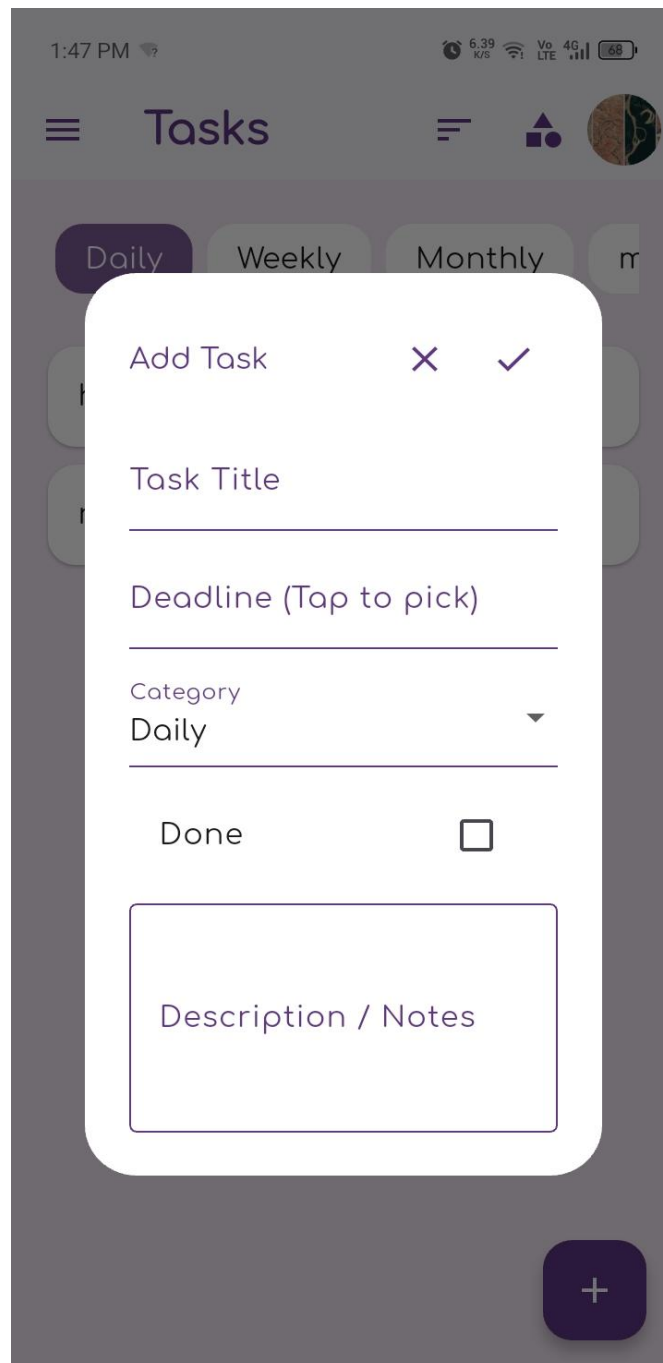


Figure 5.3.9: Add Task Dialog

- **Habit Tracker (habit_tracker.dart):** In Figure 5.3.10, the Habit Tracker page empowers users to build and rigorously monitor their daily, weekly, or monthly routines effectively. It allows the creation, editing, and deletion of custom habits paired with visual progress indicators. An interactive calendar view and streak system are employed to motivate consistent, daily user engagement.

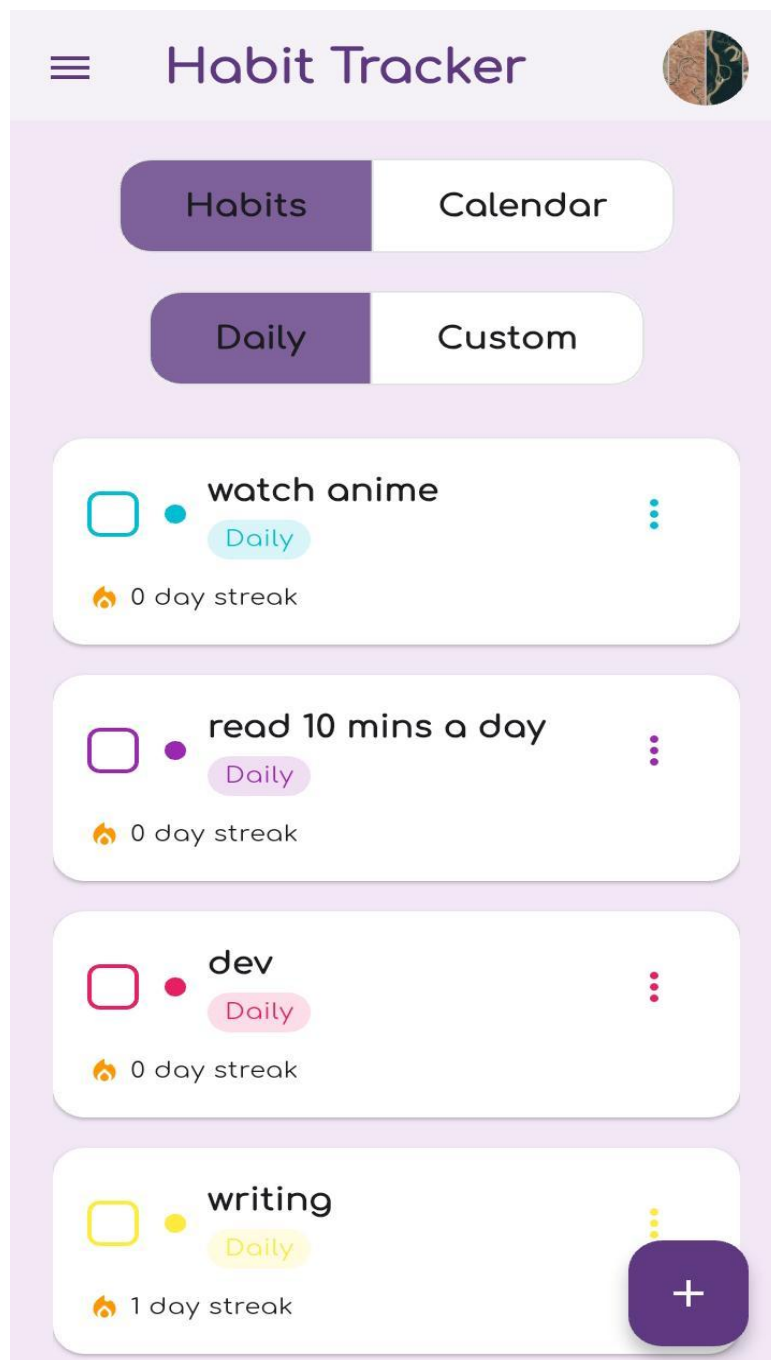


Figure 5.3.10: Habit Tracker Page

- **Add Habit:** In Figure 5.3.11, the interface of Add Habit Dialog helps users create habits by entering name, description, and selecting a color. Users can choose Daily habits or Custom habits with date ranges. A color palette is available for visual organization. After saving, the habit is added with an initial streak count of zero.

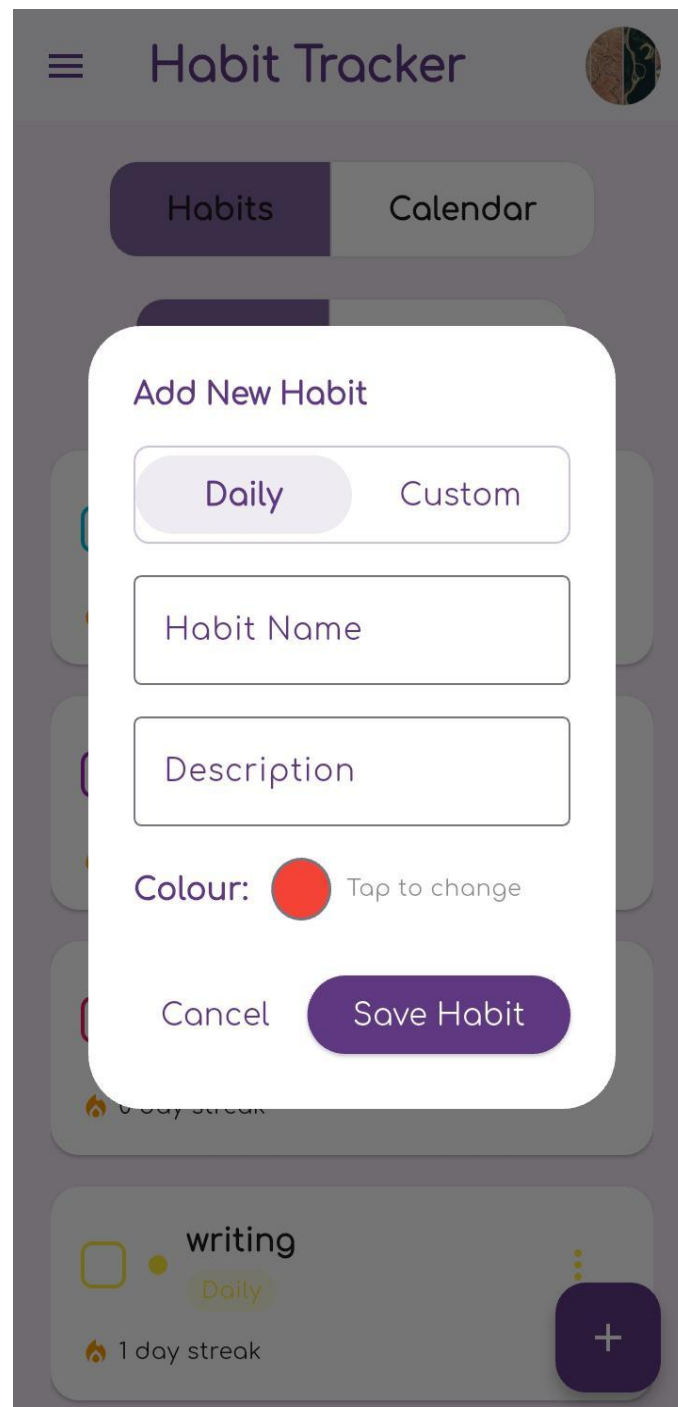


Figure 5.3.11: Add Habit Dialog

- **Habit Tracker Calendar View:** In Figure 5.3.12, the calendar view of habit tracker shows monthly habit completion history using colored indicators. Users can move between months to review consistency patterns. Selecting a date reveals habits completed on that day with streak counts. It motivates users by visually tracking progress.

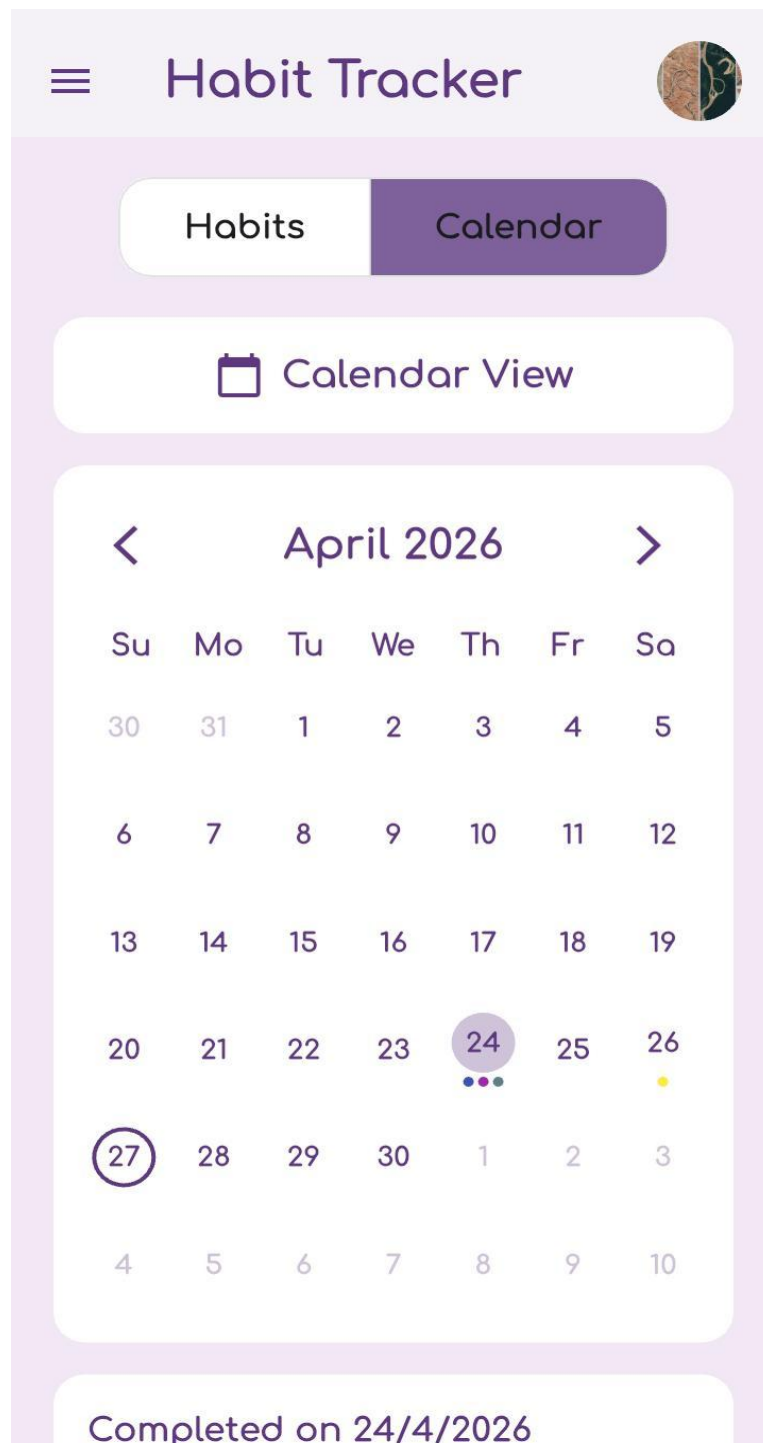


Figure 5.3.12: Habit Tracker Calendar View

- **Custom Habits (In progress):** In Figure 5.3.13, the feature of custom habits is designed for temporary habits with start and end dates. It is suitable for short term challenges such as 30-day routines or training plans. Habits are divided into In Progress and Completed tabs. Users still earn streaks by maintaining consistency during the set period.

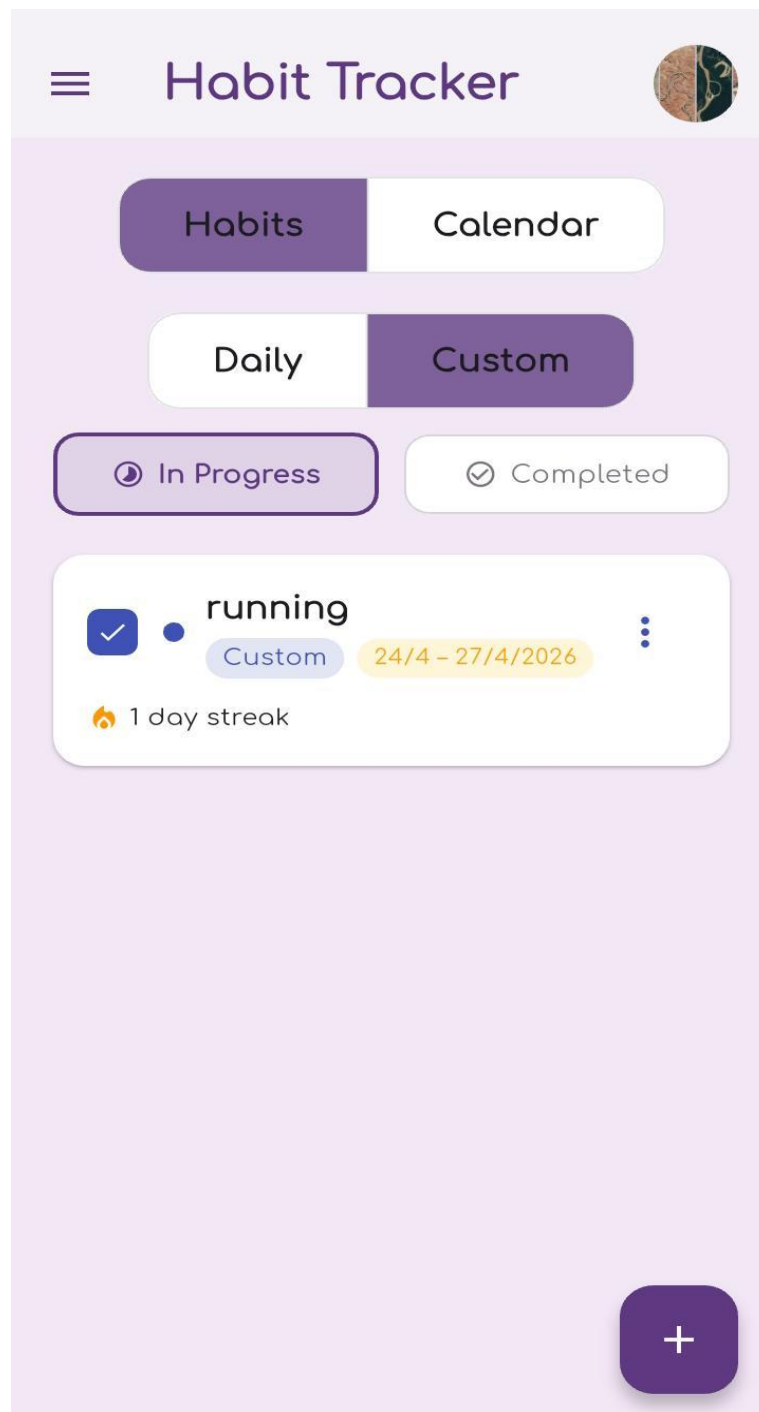


Figure 5.3.13: Habit Tracker Custom Habits

- **Notes Module (notetaking.dart):** In Figure 5.3.14, the Notes Module functions as a versatile digital notebook designed for capturing ideas, thoughts, and important data. It supports rich text organization, allowing users to neatly store and categorize their written entries. Lightning-fast retrieval of information is enabled through an accessible and highly readable layout design. It complements the broader productivity suite by keeping essential reference material close at hand.

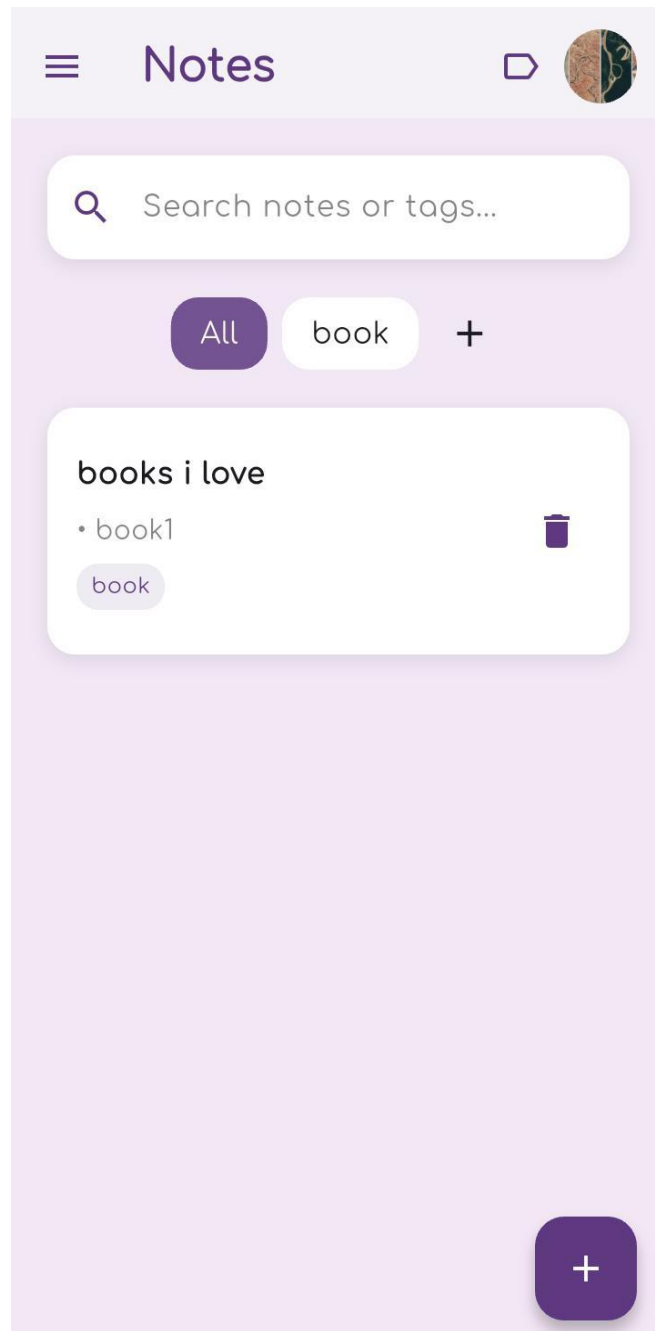


Figure 5.3.14: Notes Page

- **Edit Notes:** In Figure 5.3.15, the Edit Notes screen provides a rich editor for writing and formatting notes. Users can apply bold, italic, underline, and bullet lists for better organization. Tag options are available for adding or creating custom labels. Notes are saved to Firestore instantly with all formatting preserved.

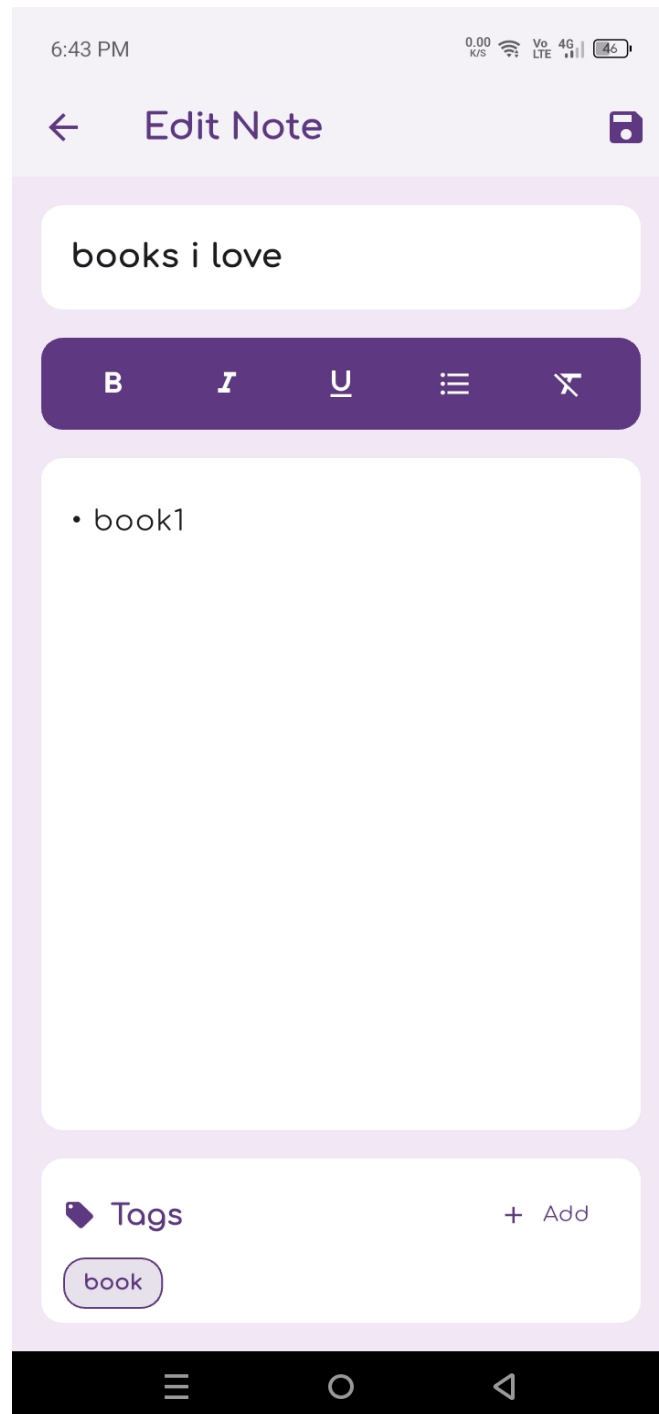


Figure 5.3.15: Note Editing View

- **AI Recommendations (airecommendation.dart):** In Figure 5.3.16, the AI Recommendations screen deeply analyzes the user's current habits and routines to generate personalized improvement suggestions. It utilizes the Gemini AI to provide highly actionable advice alongside relevant external learning resources. Suggestions are presented in an elegant, interactive UI card format that users can easily explore and adopt. It acts as a virtual productivity coach striving to continuously optimize the user's daily workflows.

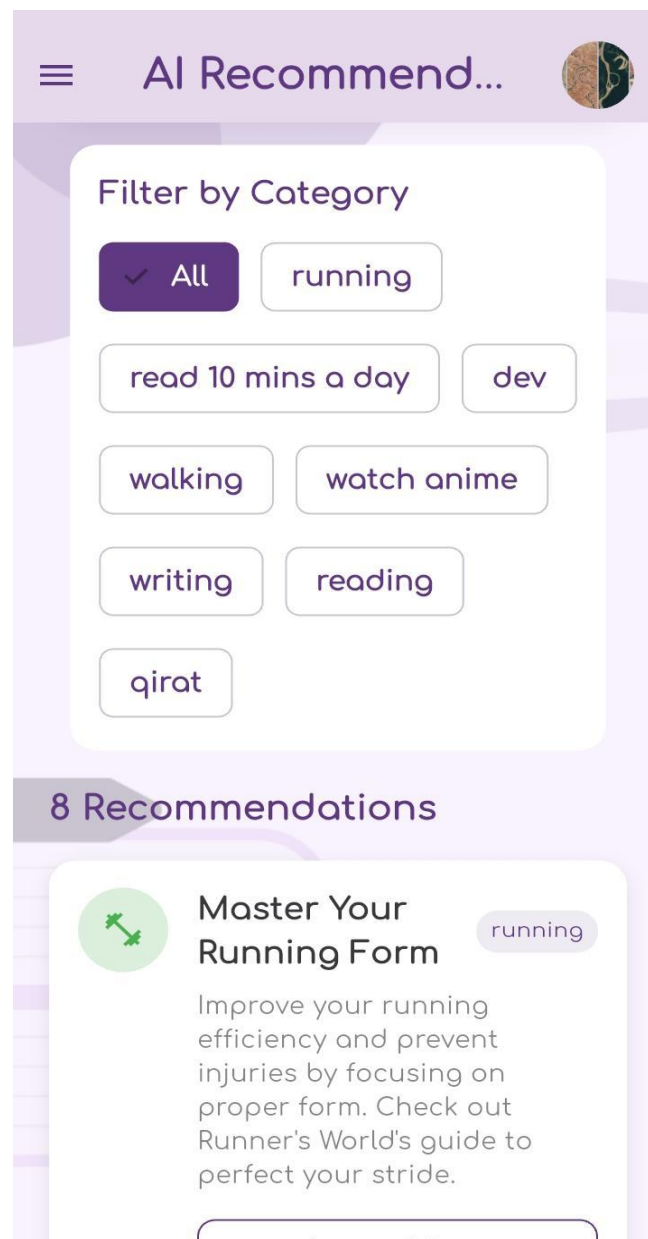


Figure 5.3.16: AI Recommendation Page

- **Focus Timer (focus_tracker.dart):** In Figure 5.3.17, the Focus Timer integrates a Pomodoro-style timer to help users maintain deep concentration during active work phases. It allows highly customizable work and break intervals tailored to suit individual productivity rhythms. The screen visually displays the remaining countdown time and accurately logs completed focus sessions for tracking. It encourages disciplined time management while actively preventing burnout through scheduled rest periods.

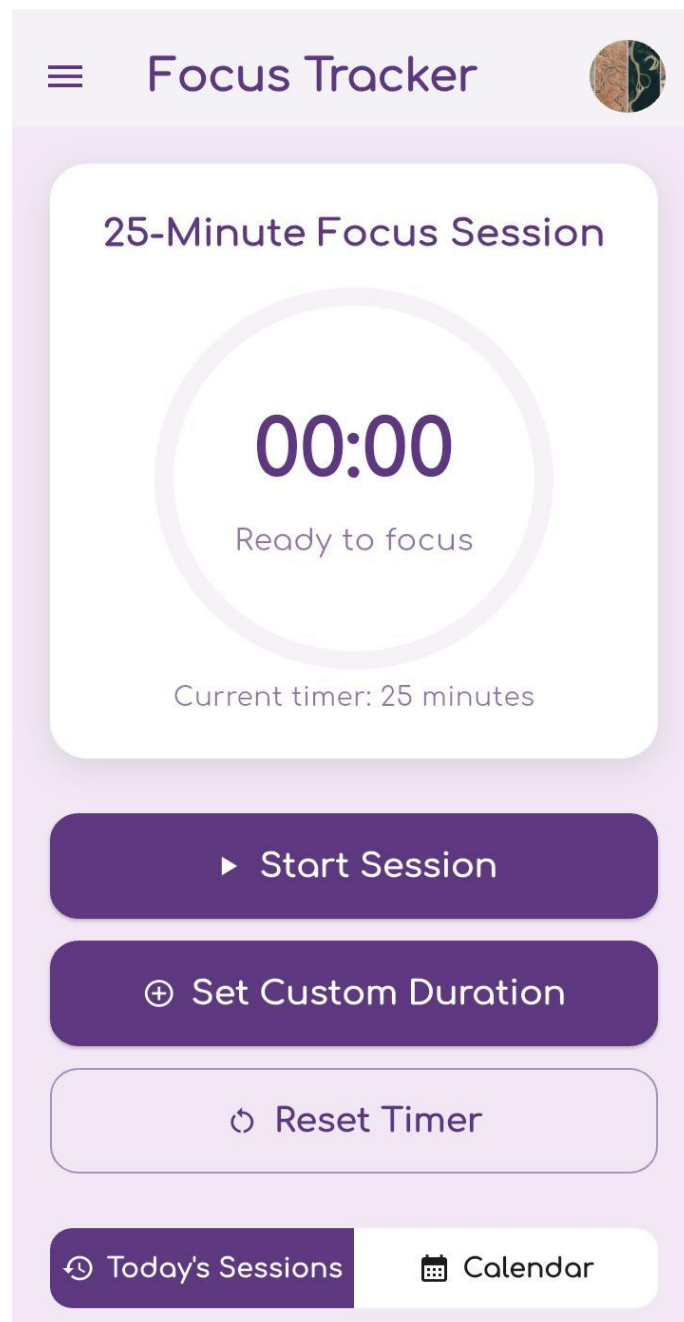


Figure 5.3.17: Focus Tracker Page

- **Focus Tracker Calendar View:** In Figure 5.3.18, the Calendar view for Focus Tracker displays past focus sessions on a monthly calendar. Dates with completed sessions are marked for easy review. Clicking on a date shows start time, end time, and total minutes focused. It helps users analyze productivity trends over time.

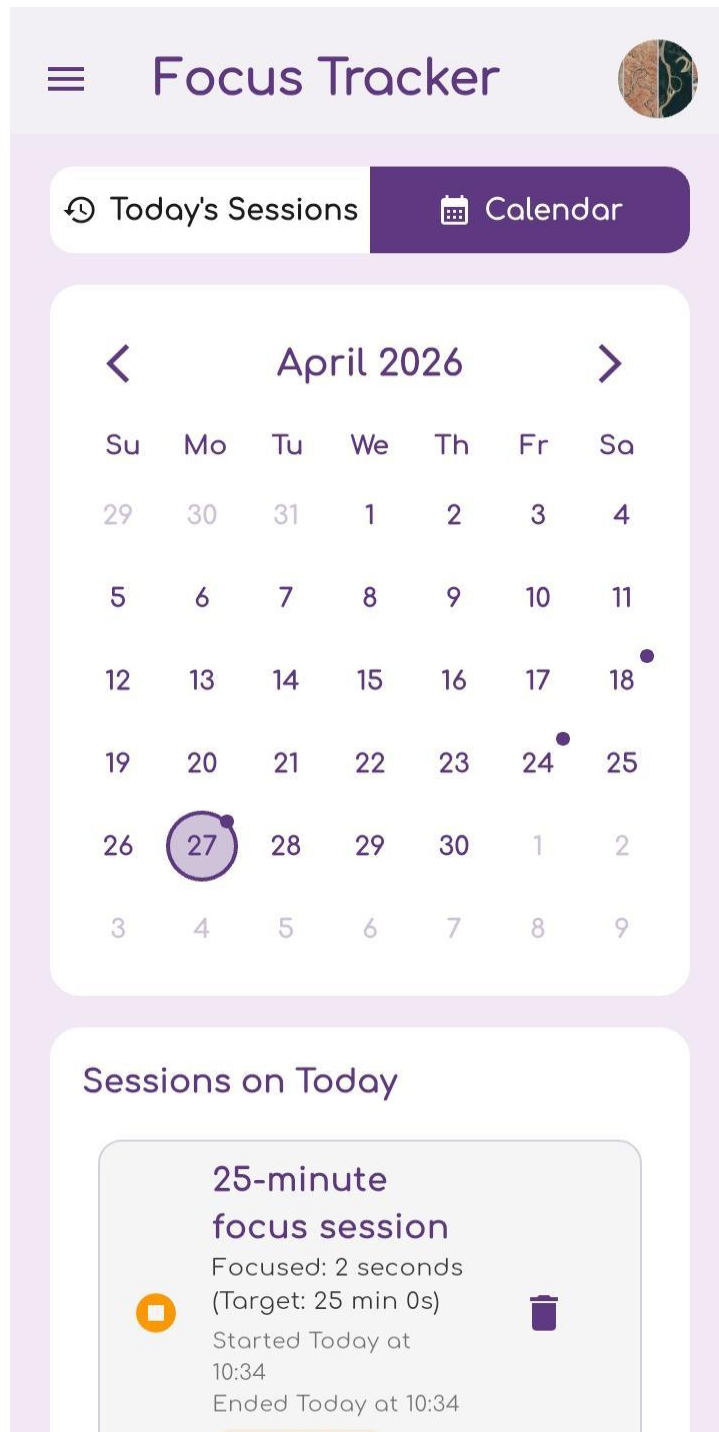


Figure 5.3.18: Focus Tracker Calendar view

- **Community & Collaboration (community.dart):** In Figure 5.3.19, the Community & Collaboration screen facilitates active social interaction by allowing users to connect, send friend requests, and converse. It enables collaborative productivity through shared workspaces and group-based task assignments. Real-time push notifications are featured to keep members updated on project changes and new messages. It transforms solitary goal-tracking routines into a shared, highly motivating team-oriented experience.

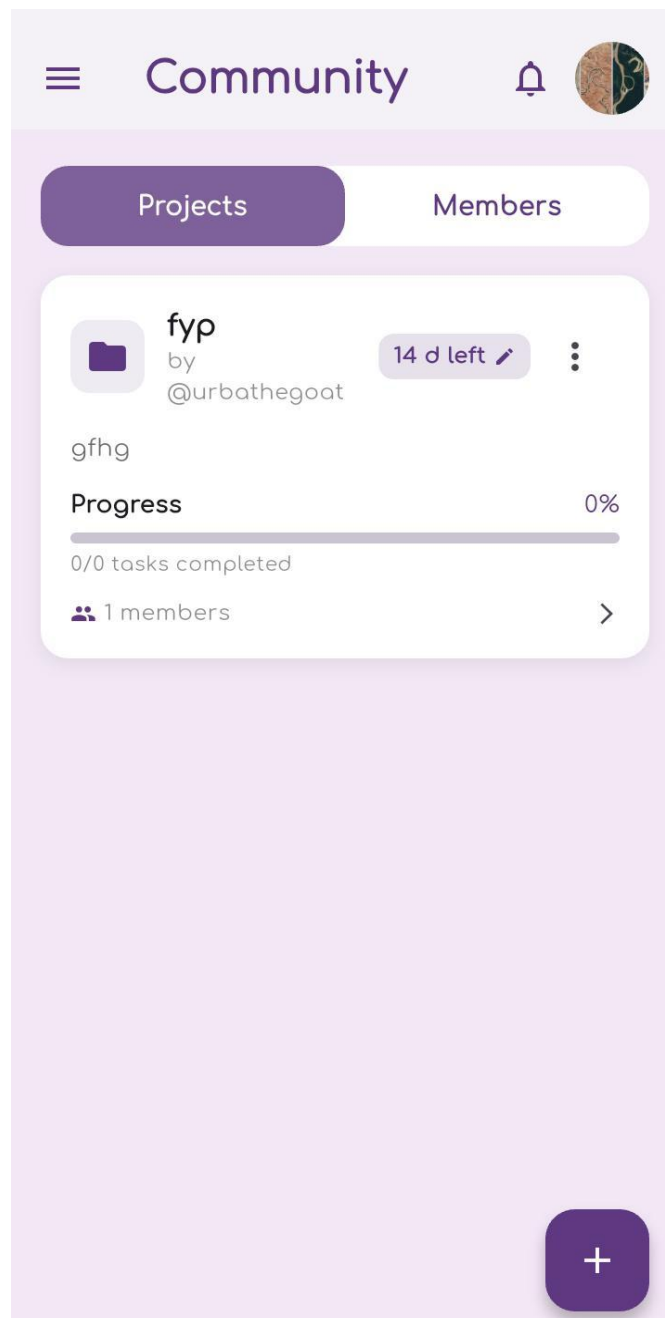


Figure 5.3.19: Community Page

- **Community Project Tasks:** In Figure 5.3.20, the Community page displays shared tasks for team or group projects. Tasks are organized into sections such as To Do, In Progress, and Done. Each task shows assigned member details and due dates. A progress bar displays project completion percentage.

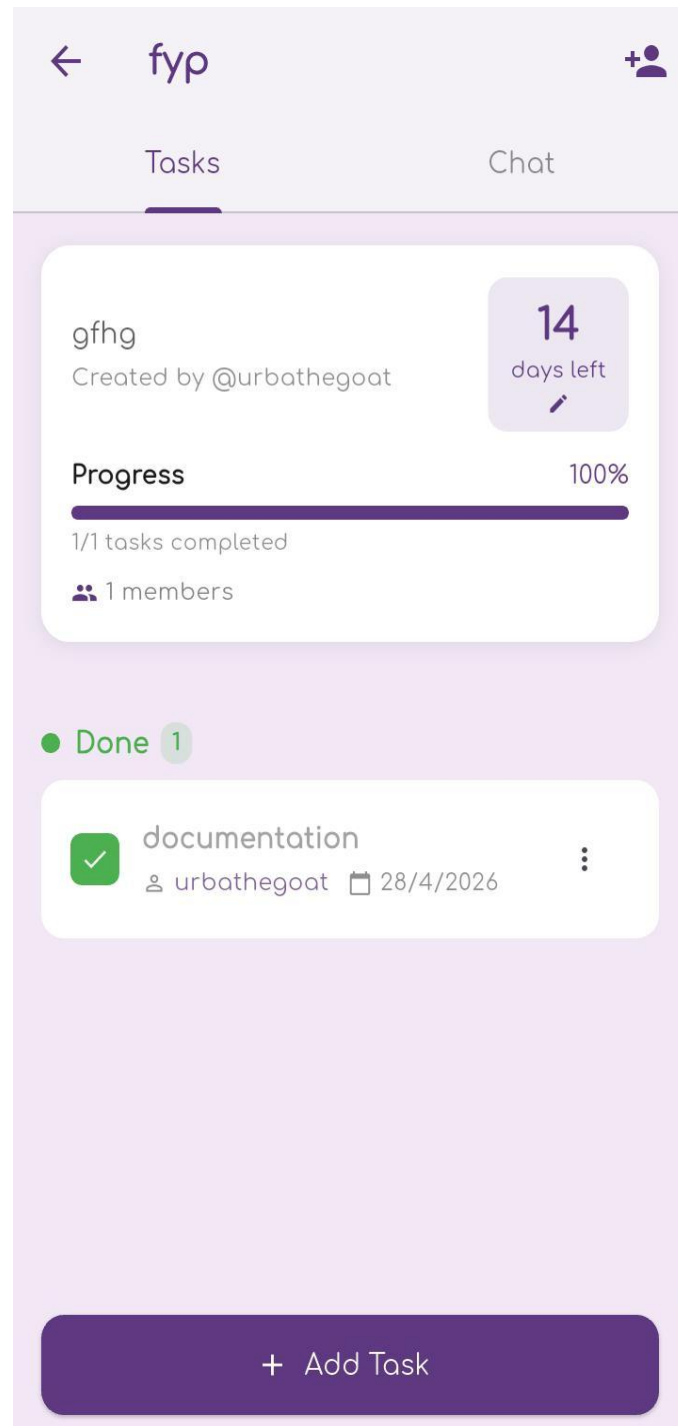


Figure 5.3.20: Community Project Tasks

- **Settings Screen (settings.dart):** In Figure 5.3.21, the Settings Screen grants user comprehensive control over their application preferences and personal system configurations. It includes quick-access options for toggling visual themes (Light/Dark mode) and managing notification alerts. Immediate access to essential account management features, including secure logout functionality, is provided. It ensures the application remains highly customizable to individual user needs and distinct tastes.

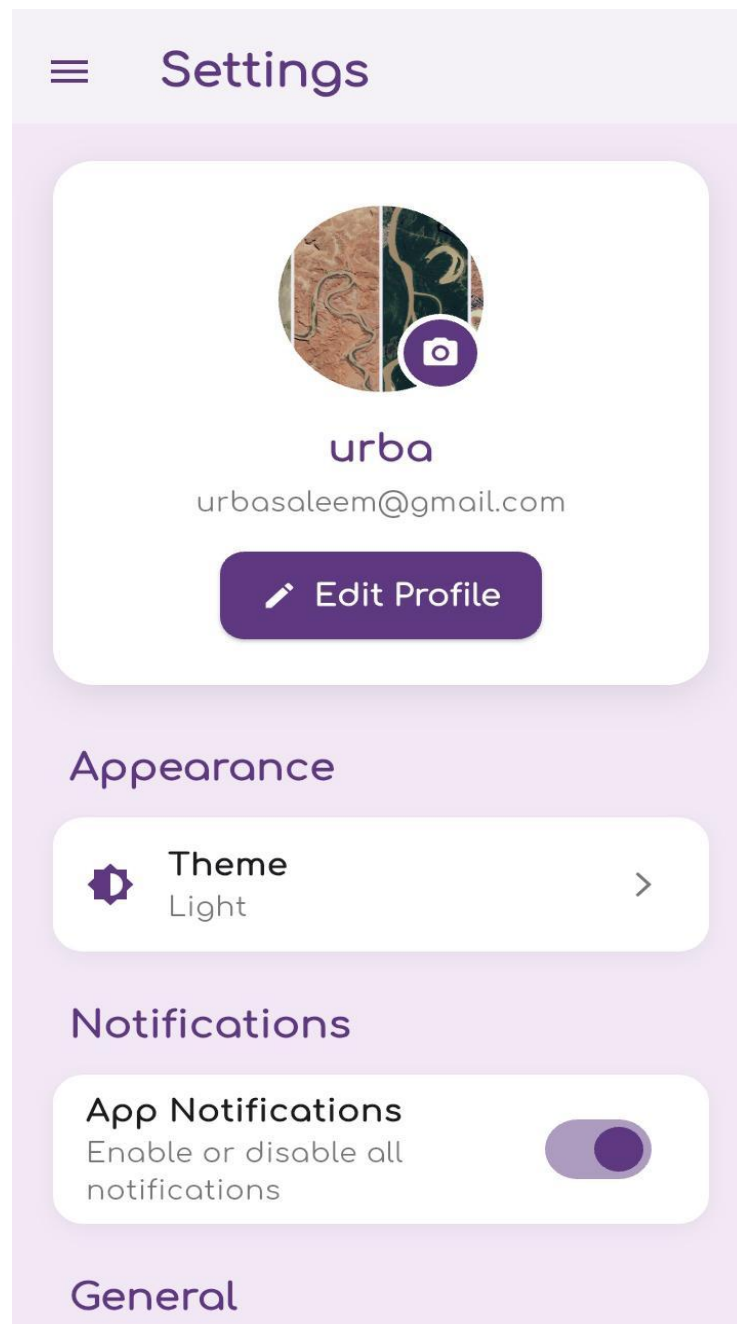


Figure 5.3.21: Settings Page

- **Dark Mode (Appearance Settings):** The feature in Figure 5.3.22 changes the app into a dark themed interface with light text. It helps reduce eye strain and saves battery on OLED devices. Users can manually enable it or match device system settings. All modules automatically adjust colors for readability.

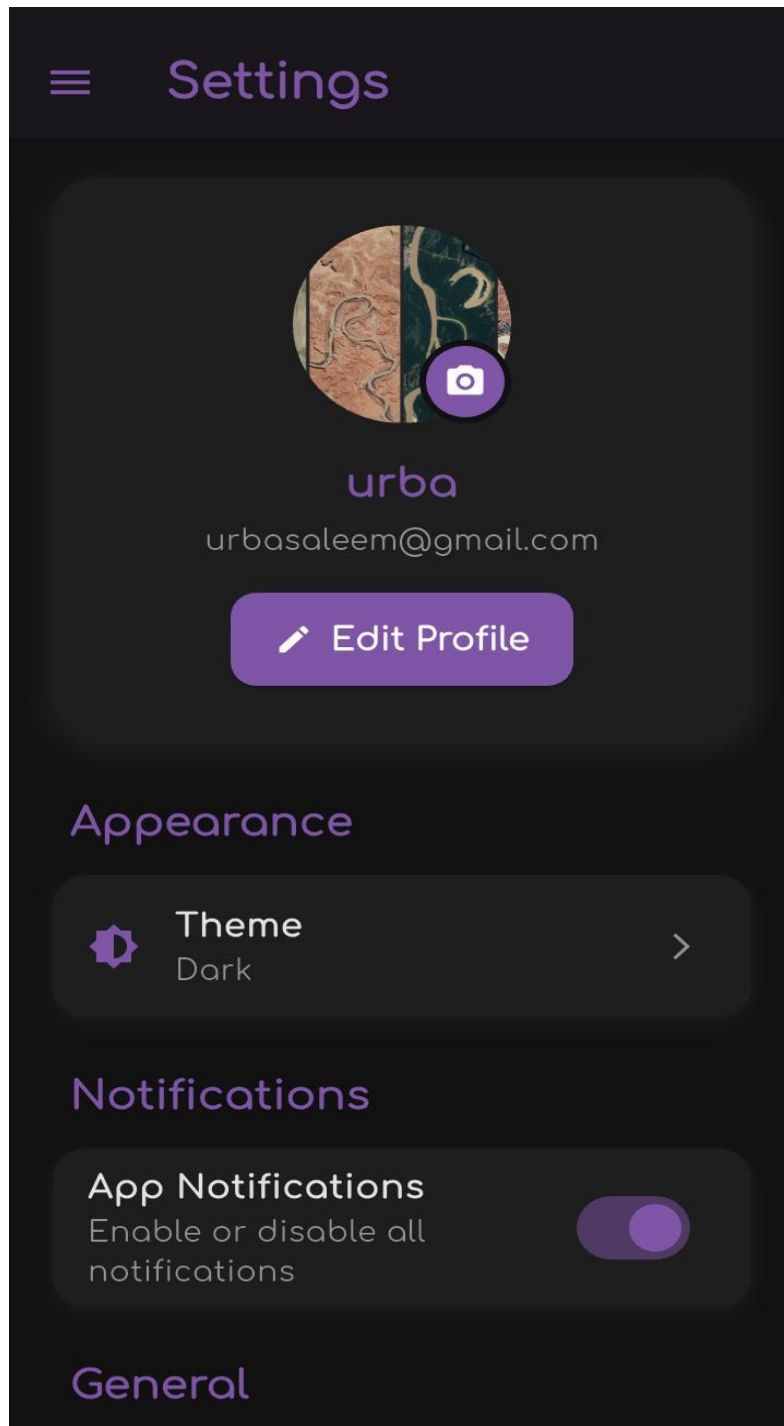


Figure 5.3.22: Dark Theme Page

Chapter 6

TESTING AND EVALUATION

2. Testing and Evaluation

This explains the testing and evaluation process of the PlanLock system. It ensures that all developed modules function correctly according to the defined requirements. Different types of testing techniques are used to identify errors, verify system performance, and ensure reliability. The goal is to confirm that the system is stable, user-friendly, and meets all functional and non-functional requirements.

6.1 Manual Testing

Manual testing was performed to verify that all features of the PlanLock application function correctly according to requirements. It ensured smooth user interaction across modules such as login, tasks, habits, notes, timer, and profile management.

6.1.1. System Testing

System testing was conducted to verify the overall functionality and stability of the PlanLock application as a complete, integrated product. It was performed using the Android Emulator within Android Studio alongside Firebase Authentication Console to simulate real-world usage conditions and monitor live responses. This phase ensured that core system-level operations such as app launch, user login, registration, password recovery, and screen navigation all behaved correctly and consistently.

Table 6.1.1: System Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status	Explanation
FR1	App Launch	Open app on Android Emulator	Welcome screen displayed	Emulator rendered welcome screen within 2 seconds	Pass	Application initializes correctly on emulator
FR1	User Login	Valid email and password via Firebase	Dashboard opens	Firebase Console showed successful token generation and UID match	Pass	Login system working properly

		Auth Console				
FR1	Invalid Login	Wrong password entered	Error message shown	Firebase Auth Console logged "INVALID_PASSWORD" rejection event	Pass	Security validation working
FR1	Sign Up	New user email and password	Account created	Firebase Auth Console displayed new UID; Firestore showed new user document	Pass	Registration successful
FR1	Reset Password	Registered email address	Reset email sent	Firebase Auth Console confirmed password reset email dispatch	Pass	Password recovery working

6.1.2 Unit Testing

Unit testing was carried out to isolate and validate the correctness of individual modules and functional components within the PlanLock application. Using the Flutter Test Framework via VS Code terminal and Dart DevTools for runtime inspection, each feature such as task creation, habit tracking, note editing, focus timing, and calendar rendering was tested independently to ensure its internal logic produces accurate and expected results. Dart DevTools provided real-time widget tree inspection and memory usage data, confirming that individual components rendered and behaved correctly in isolation.

Table 6.1.2: Unit Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status	Explanation
FR2	Create Task	Task title, description, deadline, category	Task saved to correct category	Flutter Test output: "All tests passed" — Firestore write confirmed	Pass	Task module logic verified

FR2	Add Task Dialog	Open dialog, fill all fields, tap Save	Task added with deadline and category	Dart DevTools widget tree showed dialog dismissed and list updated	Pass	Add Task dialog functional
FR3	Add Habit	Habit name, color selection, daily frequency	Habit added with zero streak	Flutter Test output: "Habit creation test passed" — streak initialized at 0	Pass	Habit tracker functional
FR3	Custom Habit	Start date, end date, habit name	Custom habit created with duration	Flutter Test confirmed habit appears under "In Progress" tab correctly	Pass	Custom habit logic working
FR7	Save Note	Text content and tag input	Note saved with formatting and tags	Flutter Test output: "Note save test passed" — Firestore document created	Pass	Notes module working
FR7	Edit Note	Modify bold text and add new tag	Formatted note updated in Firestore	Dart DevTools confirmed widget re-render and Firestore write success	Pass	Note editor functional
FR6	Start Timer	Tap Start Focus Session button	Timer countdown begins	Flutter Test output: "Timer initialization test passed" — countdown verified	Pass	Focus timer working
FR3, FR4	Habit Calendar View	Open calendar tab after habit completion	Monthly dots displayed on correct dates	Dart DevTools widget tree confirmed calendar cells marked correctly	Pass	Habit calendar view working

FR6	Focus Calendar View	Open focus history after session completion	Monthly session grid displayed	Flutter Test output: "Session log render test passed" — entries visible	Pass	Focus history calendar functional
-----	---------------------	---	--------------------------------	---	------	-----------------------------------

6.1.3 Functional Testing

Functional testing was performed to confirm that all individual features and system capabilities of PlanLock behave exactly as defined in the functional requirements. This phase used Firebase Firestore Console to monitor live data changes, the Groq AI API Dashboard to verify response generation, and Android Emulator to observe real-time UI behavior across every major screen and user flow. The goal was to ensure that each feature delivers the correct output and user experience when operated as part of the full running application.

Table 6.1.3: Functional Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status	Explanation
FR2	Task Completion	Tap checkmark on task item	Task marked as done and updated	Firestore Console showed "completed: true" field updated instantly	Pass	Task lifecycle correct
FR3	Habit Tracking	Mark daily habit as complete	Streak count incremented by 1	Firestore Console confirmed streak field incremented correctly	Pass	Habit logic working
FR3, FR4	Custom Habit Completion	Complete habit within defined date range	Streak incremented and habit moved to Completed tab	Firestore updated status field; Emulator showed habit in Completed tab	Pass	Custom habit lifecycle correct

FR5	AI Recommendation	Open AI Recommendations screen	Personalized suggestions displayed	Gemini AI Dashboard logged successful API call; cards rendered on screen	Pass	AI system functional
FR6	Focus Session Logging	Complete a full focus timer session	Session recorded with start and end time	Firestore Console showed new session document with timestamps logged	Pass	Focus session history accurate
FR7	Note Tag Filtering	Select a tag from the horizontal tag bar	Only notes with selected tag displayed	Emulator filtered note list correctly matching selected label	Pass	Tag system functional
FR8	Community Post	Submit a new post in community module	Post visible to all group members	Firebase Realtime Database Console showed new post node created instantly	Pass	Collaboration working

6.1.4 Integration Testing

Integration testing was conducted to verify that the various modules and services within PlanLock communicate and function correctly when combined as a unified system. This phase used Firebase Firestore and Realtime Database Console to trace data flow across modules, Gemini AI API Dashboard to monitor cross-module AI interactions, and Android Emulator to observe complete user journeys from start to finish across interconnected screens. Key workflows such as logging in and immediately creating a task, completing a habit and seeing it reflected on the calendar, or finishing a focus session and reviewing it in the history view were all tested in sequence. The results confirmed that cross-module interactions are seamless and data remains consistent throughout the entire system.

Table 6.1.4: Integration Testing

FR No	Test Case	Input	Expected Result	Actual Result	Status	Explanation
FR1, FR2	Login → Task Creation	Login then add task via Add Task dialog	Task linked to authenticated user with category and deadline	Firestore showed task document created under correct user UID	Pass	Auth and task modules integrated properly
FR1, FR3	Login → Habit Update	Login then complete a daily or custom habit	Streak updated and calendar dot marked for today	Firebase updated streak field; Emulator showed dot on today's calendar date	Pass	Auth and habit data flow correct
FR2, FR6	Task + Timer	Start focus session associated with active task	Time recorded and session logged under task	Firestore showed new session document linked correctly to task ID	Pass	Task and timer integration successful
FR5, FR7	AI + Notes	Open AI screen then save a recommendation as note	Note stored in Notes module with relevant tag	Firestore confirmed note created; Gemini Dashboard logged successful suggestion API call	Pass	AI and notes cross-module working

FR3, FR4	Habit Completion → Calendar	Complete habit then open habit calendar view	Completion dot appears on today's date in calendar	Firestore updated completion record; Emulator confirmed dot rendered on correct date	Pass	Habit and calendar sync working
FR6	Focus Session → Calendar	Complete timer session then open focus history	Session log entry appears on today's date	Firestore showed session document; Emulator confirmed entry visible in calendar grid	Pass	Focus tracker and calendar integrated

6.2 Automated Testing

Automated testing was implemented to systematically validate the stability, correctness, and performance of PlanLock's modules without requiring manual intervention for every test run. The testing suite utilized the Flutter Test Framework executed via VS Code terminal for unit and widget-level automation, the Firebase Emulator Suite for safely simulating backend cloud behavior in a fully controlled local environment, Dart DevTools for performance and widget inspection, and the Groq API Dashboard to verify intelligent response generation across recommendation modules. Automated tests were designed to cover all critical paths including authentication flows, task and habit logic, UI component rendering, timer functionality, and theme switching. This approach significantly reduced testing time, ensured consistent repeatability across builds, and provided the team with reliable, detailed feedback on the system's overall health after each development iteration.

Table 6.2: Automated Testing

FR No	Tool Used	Test Case	Expected Result	Actual Result	Status	Explanation
FR1	Firebase Emulator Suite	Login authentication test	Authentication success and token generated	Emulator Suite output: "AUTH_SUCCESS — UID generated and session token issued"	Pass	Backend auth flow verified locally
FR1	Firebase Emulator Suite	Reset password flow test	Reset email trigger confirmed	Emulator Suite output: "PASSWORD_RESET_EMAIL_SENT — event logged successfully"	Pass	Recovery flow verified without live email
FR2	Flutter Test Framework	Task creation via Add Task dialog	Task saved with correct category and deadline	Terminal output: "Task creation test passed in 112ms"	Pass	Task dialog logic correct
FR3	Flutter Test Framework	Habit UI rendering with color and frequency	Habit card displayed with correct color and label	Terminal output: "Habit widget render test passed in 98ms"	Pass	Habit UI validated
FR3, FR4	Flutter Test Framework	Custom habit date range and tab logic	Habit appears under In Progress tab within date range	Terminal output: "Custom habit logic test passed in 134ms"	Pass	Custom habit date logic verified
FR6	Flutter Test Framework	Focus timer session start and logging	Timer runs and session document created in Firestore	Terminal output: "Timer session test passed in 205ms"	Pass	Timer and logging verified
FR7	Dart DevTools	Note editor with bold	Formatted note saved correctly with tag	DevTools widget inspector confirmed correct	Pass	Note editor UI and save

		formatting and tag assignment		state update and Firestore write		logic validated
FR3, FR4	Dart DevTools	Habit calendar dot rendering after completion	Colored dots appear on correct completed dates	DevTools widget tree confirmed CalendarCell widgets marked with completion state	Pass	Calendar UI rendering validated
FR1	Firebase Emulator Suite	Login authentication test	Authentication success and token generated	Emulator Suite output: "AUTH_SUCCESS — UID generated and session token issued"	Pass	Backend auth flow verified locally

6.3 Tools Used

The tools selected for testing PlanLock were chosen to provide comprehensive coverage across the frontend, backend, and AI service layers of the application. Each tool was specifically matched to the type of validation required, with online and emulator-based tools such as Firebase Console, Firebase Emulator Suite, Dart DevTools, Groq AI API Dashboard, and the Flutter Test Framework collectively ensuring that both functional correctness and technical reliability were thoroughly assessed. Together, these tools formed a robust testing ecosystem that supported manual verification, automated regression testing, live database monitoring, and real-time cloud behavior simulation, enabling the team to deliver a stable and high-quality final application.

Table 6.3: Testing Tools and Their Usage

Tool Name	Purpose	Tested Modules	Result	Explanation
Android Emulator	Visual app execution and UI testing	Full application — all screens	Pass	App runs smoothly and renders

(Android Studio)				correctly across all screens
Firebase Authentication Console	Live user login, sign up, and reset password monitoring	FR1 – Authentication, Reset Password	Pass	Auth events and tokens verified via Firebase Console logs
Firebase Firestore Console	Real-time database read and write verification	FR2, FR3, FR7– Tasks, Habits, Notes, Profile	Pass	All data stored, updated, and synced correctly in Firestore
Firebase Realtime Database Console	Live community post and collaboration data monitoring	FR8 – Community and Collaboration	Pass	Posts and group interactions updated in real time
Firebase Emulator Suite	Local backend simulation without affecting live data	FR1, FR2, FR3 – Auth, Tasks, Habits	Pass	Cloud behavior safely tested in controlled local environment
Flutter Test Framework	Automated unit and widget testing via VS Code terminal	FR2, FR3, FR4, FR6, FR7 – All core modules	Pass	Terminal output confirmed all test cases passed with ms timing
Dart DevTools	Widget tree inspection and runtime performance analysis	FR3, FR4, FR7 – Habits, Calendar, Notes Editor	Pass	Widget states and re-renders confirmed correct via inspector
Grok AI API Dashboard	AI response monitoring, latency, and token usage tracking	FR5– AI Recommendations	Pass	Dashboard confirmed 200 OK responses with accurate AI output

Chapter 7

CONCLUSION AND FUTURE WORK

3. Conclusion and Future Work

7.1 Conclusion

PlanLock was developed as an intelligent all-in-one productivity application that combines task management, habit tracking, focus sessions, note taking, community collaboration, and AI recommendations into a single unified platform. The project successfully solves the common problem of using multiple separate applications for productivity needs by integrating all essential modules into one cohesive system, providing users with a smoother, more organized, and efficient daily experience. The application was implemented using Flutter for cross-platform support, Firebase for authentication and real-time cloud data storage, and the Groq AI API for personalized productivity recommendations. Features such as streak tracking with achievement levels and badges, custom and daily habit monitoring with calendar views, Pomodoro-style focus timer sessions with session history, rich text note taking with tag-based organization, and community-based collaborative project task management collectively enhance user motivation and long-term productivity. Testing results across system, unit, functional, integration, and automated testing phases confirmed that all modules perform reliably and meet the defined functional and non-functional requirements. Overall, PlanLock demonstrates how modern mobile technologies, cloud infrastructure, and AI-powered assistance can be combined into one centralized digital solution to support personal growth, time management, and team collaboration.

7.2 Future Work

Although PlanLock delivers a comprehensive productivity solution with its current feature set, several meaningful enhancements can be introduced in future versions to further elevate the user experience and system capability. A dedicated XP and leveling system can be expanded beyond the current streak-based achievement levels, allowing users to earn experience points specifically for completing tasks, focus sessions, and collaborative activities, unlocking higher ranks, exclusive rewards, and personalized milestones for sustained long-term motivation. An SQLite offline database can be integrated to allow users to access their tasks, habits, and notes without an active internet connection, with automatic background synchronization to Firebase once connectivity is restored, significantly improving reliability and usability in low-network environments.

An AI API can be used to integrate an intelligent chatbot in the app for app-related guidance or generic intelligent responses to queries. Voice command support can be introduced to allow users to add tasks, start focus sessions, and query the chatbot hands-free, making the application more accessible and convenient during multitasking scenarios. Advanced analytics dashboards can be developed to give users deeper visual insights into their weekly and monthly productivity trends across habits, tasks, and focus sessions beyond the current calendar and streak views. Wearable device integration, such as smartwatch support, can allow users to receive reminders, log habit completions, and monitor focus sessions directly from their wrist without opening the app. Smart calendar synchronization with external platforms such as Google Calendar can allow tasks and deadlines set within PlanLock to automatically reflect in the user's broader schedule. Multilingual support can be added to make the application accessible to a wider global audience across different languages and regions. Team collaboration within the community module can be further enhanced with real-time chat messaging, file sharing, and structured project management tools beyond the current shared task boards. These improvements would transform PlanLock into an even more powerful, inclusive, and intelligent productivity ecosystem capable of serving a broader range of personal and professional user needs.

8. References

- [1] [Online]. Available: <https://firebase.google.com/docs>.
- [2] "Todoist: the to-do list to organize work and life," Doist, [Online]. Available: <https://todoist.com/>.
- [3] [Online]. Available: <https://www.notion.com/templates/search?query=task+management..>
- [4] [Online]. Available: <https://firebase.google.com/docs>.
- [5] [Online]. Available: <https://docs.flutter.dev/get-started/flutter-for/android-devs>.
- [6] [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>.
- [7] [Online]. Available: <https://developer.android.com/jetpack/guide>.
- [8] [Online]. Available: <https://www.nngroup.com/articles/ten-usability-heuristics/>.